

Java Swing e Banco de Dados

Fabício Tonetto Londero

Java Swing

- O Java Swing é uma biblioteca gráfica fornecida pela linguagem de programação Java para criar interfaces gráficas de usuário (GUIs).
- Ele permite criar aplicativos com uma aparência e comportamento visual atraentes e interativos.
- O Swing é parte da Java Foundation Classes (JFC) e oferece uma ampla gama de componentes GUI, como botões, caixas de texto, painéis e janelas, que podem ser usados para construir interfaces ricas e dinâmicas.

Java Swing

- Vantagens
 - Plataforma Independente: As aplicações Swing funcionam em várias plataformas sem a necessidade de reescrever o código.
 - Componentes Personalizáveis: É possível personalizar a aparência e o comportamento dos componentes Swing para atender às necessidades específicas do projeto.
 - Rica Variedade de Componentes: Swing oferece uma vasta gama de componentes que podem ser combinados para criar interfaces complexas.
 - Interface Responsiva: As aplicações Swing são conhecidas por serem responsivas e rápidas.
 - Look and Feel Consistente: Swing permite aplicar diferentes "Look and Feels" para adaptar a aparência da aplicação ao sistema operacional.

Java Swing

- Desvantagens
 - Aparência e Experiência de Usuário: O Java Swing não oferece uma aparência nativa em todas as plataformas. Isso significa que, dependendo do sistema operacional, as interfaces criadas com Swing podem parecer deslocadas e diferentes das interfaces nativas, o que pode afetar a experiência do usuário.
 - Desempenho: O Java Swing pode ser mais pesado em termos de recursos e desempenho em comparação com tecnologias de GUI mais modernas. Isso pode levar a uma menor responsividade e atrasos perceptíveis em interfaces mais complexas.
 - Falta de Recursos Modernos: O Swing carece de recursos modernos de design e interação presentes em outras bibliotecas e frameworks mais recentes. Isso pode limitar a capacidade de criar interfaces altamente interativas e visualmente atraentes.
- Entre outras...

Java Swing

- Porém, o Java Swing é um método que facilmente poderemos compreender o desenvolvimento de software
- Utilizar uma conexão com banco de dados para a integração e o desenvolvimento de software.
- Utilizar componentes e interação com o usuário.
- Entre outros pontos...

Java Swing

- Existem diversos componentes para serem utilizados numa interface gráfica utilizando Swing
 - **Jpanel**: um componente que serve como um contêiner leve e flexível para agrupar outros componentes. Ele não possui uma aparência própria, mas é utilizado principalmente para organizar e agrupar outros componentes, como botões, rótulos, campos de texto, entre outros, em uma área comum dentro de uma interface gráfica.
 - **JButton**: Um botão clicável que executa uma ação quando pressionado.
 - **JLabel**: Um componente que exibe um texto ou uma imagem.
 - **JTextField**: Uma caixa de texto onde os usuários podem inserir texto.

Java Swing

- **JCheckBox:** Uma caixa de seleção que permite aos usuários escolher entre duas opções (marcada ou desmarcada).
- **JRadioButton:** Botões de opção que permitem aos usuários escolher uma única opção de um grupo.
- **TextArea:** Componente que fornece uma área de texto multilinha, permitindo que os usuários insiram, visualizem e editem texto em um formato que vai além das limitações de um único campo de texto, como o `TextField`.
- **Jtable:** Componente que oferece a capacidade de exibir e editar dados em forma de tabela. Ela é frequentemente utilizada para exibir informações tabulares, como listas, tabelas de dados, planilhas e outras estruturas similares.

Java Swing

- O padrão de nomenclatura de componentes no Java Swing segue convenções que ajudam a tornar o código mais legível, compreensível e organizado.
- Essas convenções facilitam a identificação dos tipos de componentes e sua finalidade dentro da interface gráfica.
- A convenção geral para nomear componentes no Java Swing é a seguinte:
 - O nome do componente começa com um prefixo que indica o tipo de componente:
 - btn para botões (JButton)
 - lbl para rótulos (JLabel)

Java Swing

- O nome do componente começa com um prefixo que indica o tipo de componente:
 - txt para campos de texto (JTextField)
 - cmb para caixas de seleção (JComboBox)
 - chk para caixas de seleção (JCheckBox)
 - rad para botões de opção (JRadioButton)
 - tbl para tabelas (JTable)
 - pnl para painéis (JPanel)

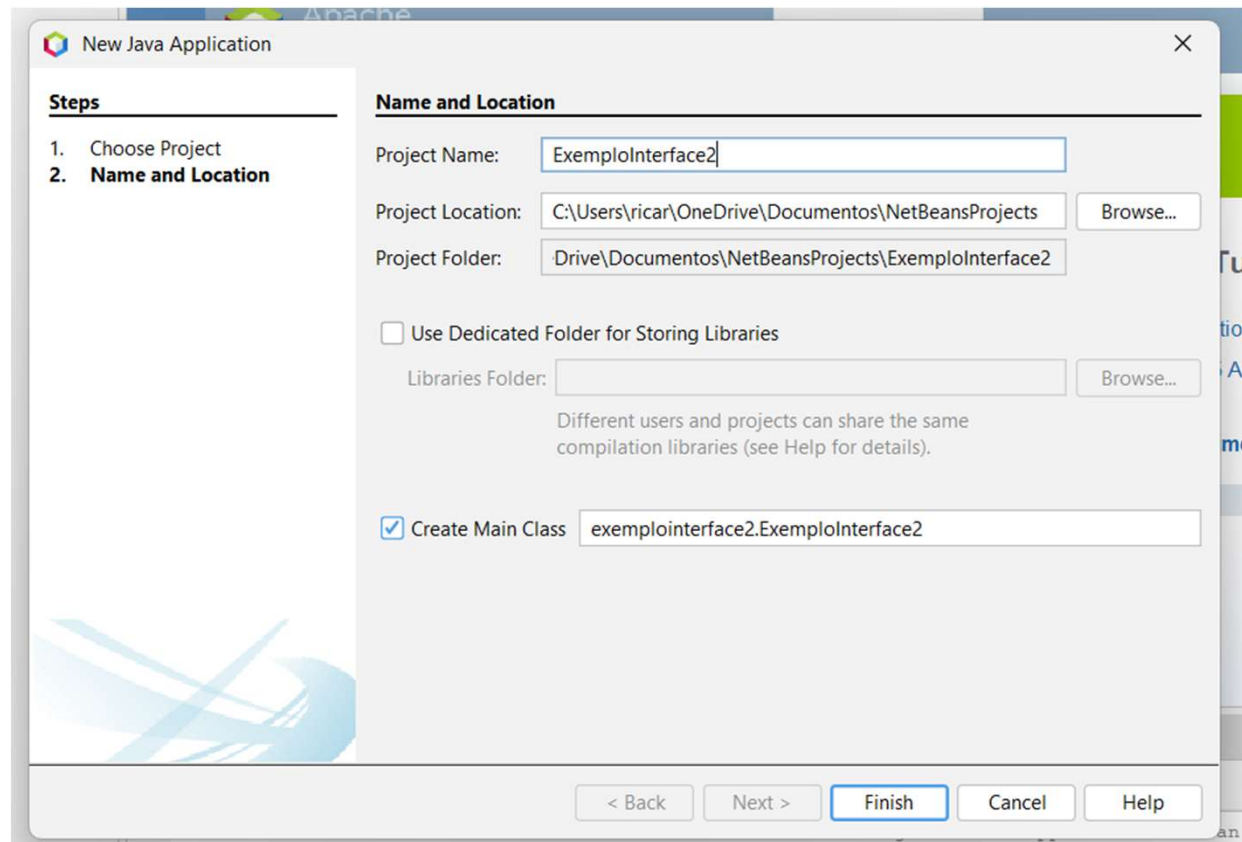
Java Swing

- Após o prefixo, você pode usar um nome descritivo para indicar a função ou finalidade do componente.
- Por exemplo: btnSalvar, lblMensagem, txtNome, cmbCidades.

Java Swing

- Agora, abra o NetBeans
 - Clique em “File” -> “New Project”
 - Selecione “Java with Ant” e “Java Application”
 - Defina o nome para o projeto, por exemplo, ExemploInterface2 e desmarque a opção “Create Main Class”

Java Swing



The screenshot shows the 'New Java Application' dialog box in the NetBeans IDE. The dialog is titled 'New Java Application' and has a close button (X) in the top right corner. On the left side, there is a 'Steps' panel with two steps: '1. Choose Project' and '2. Name and Location'. The 'Name and Location' step is currently selected. The main area of the dialog is titled 'Name and Location' and contains the following fields and options:

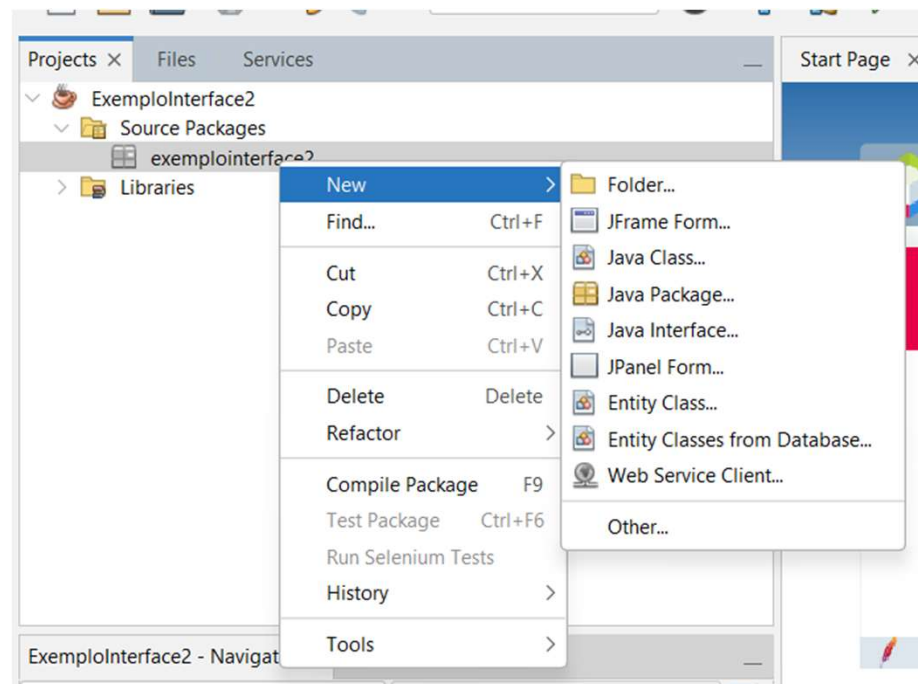
- Project Name:** A text field containing 'ExemplInterface2'.
- Project Location:** A text field containing 'C:\Users\ricar\OneDrive\Documentos\NetBeansProjects' and a 'Browse...' button.
- Project Folder:** A text field containing 'Drive\Documentos\NetBeansProjects\ExemplInterface2'.
- Use Dedicated Folder for Storing Libraries:** An unchecked checkbox.
- Libraries Folder:** A text field and a 'Browse...' button.
- Create Main Class:** A checked checkbox, followed by a text field containing 'exemplinterface2.ExemplInterface2'.

At the bottom of the dialog, there are five buttons: '< Back', 'Next >', 'Finish' (highlighted with a blue border), 'Cancel', and 'Help'.

Java Swing

- Clique com botão direito no nome do pacote
 - “New” -> “Jframe Form”

Java Swing



Java Swing

New JFrame Form

Steps

1. Choose File Type
2. **Name and Location**

Name and Location

Class Name:

Project:

Location:

Package:

Created File:

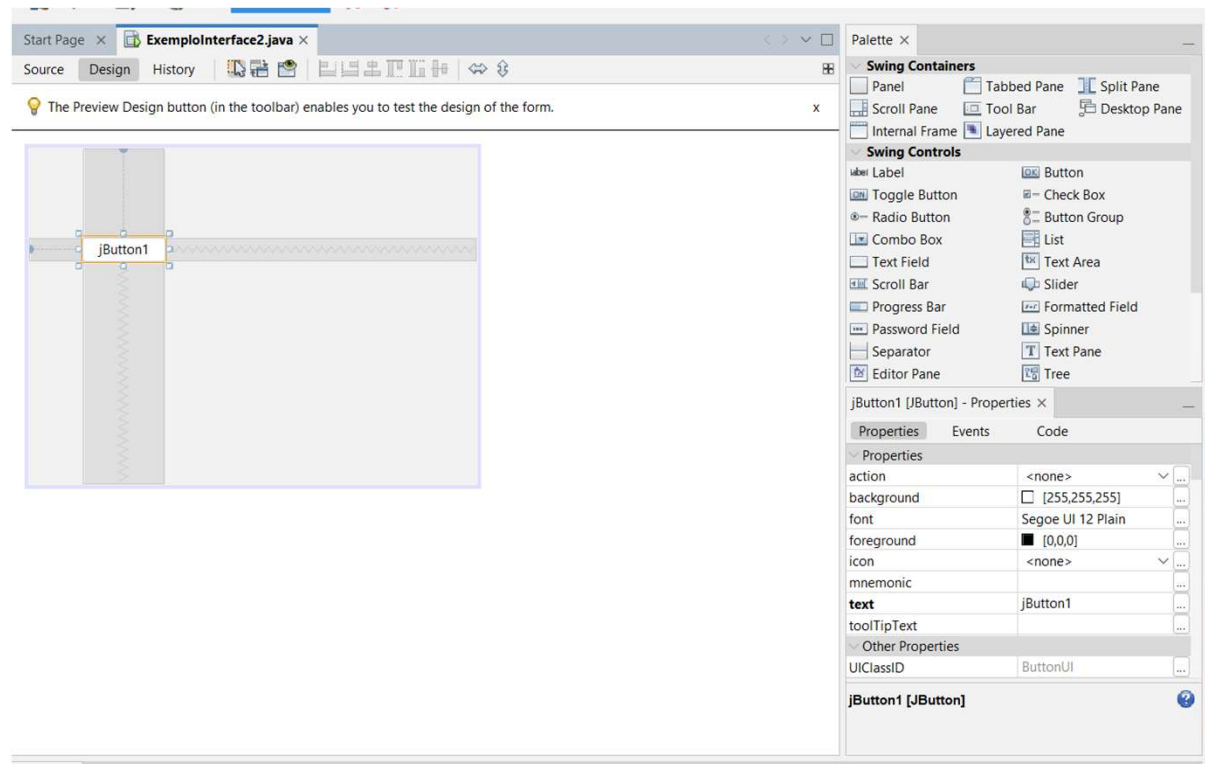
Superclass:

Interfaces:

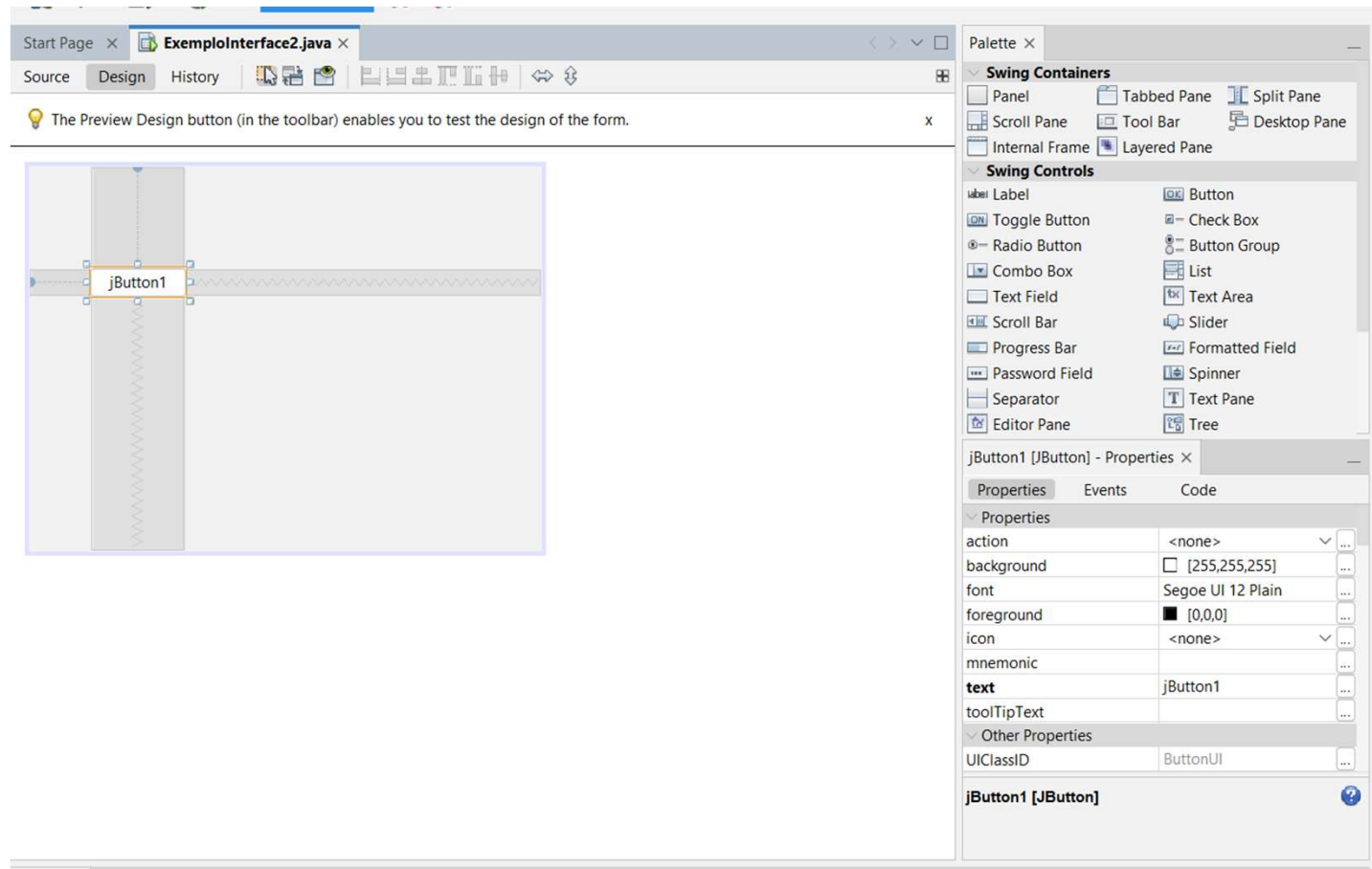
< Back Next > **Finish** Cancel Help

Java Swing

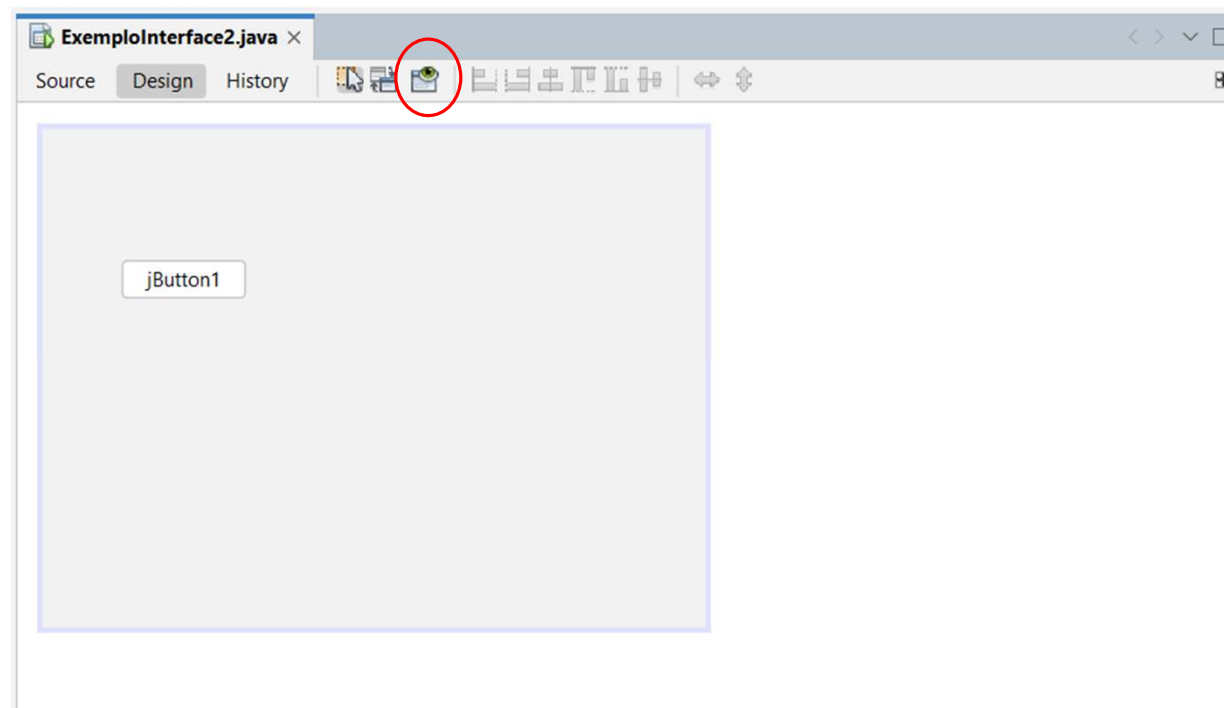
- Agora, adicione um botão no jForm



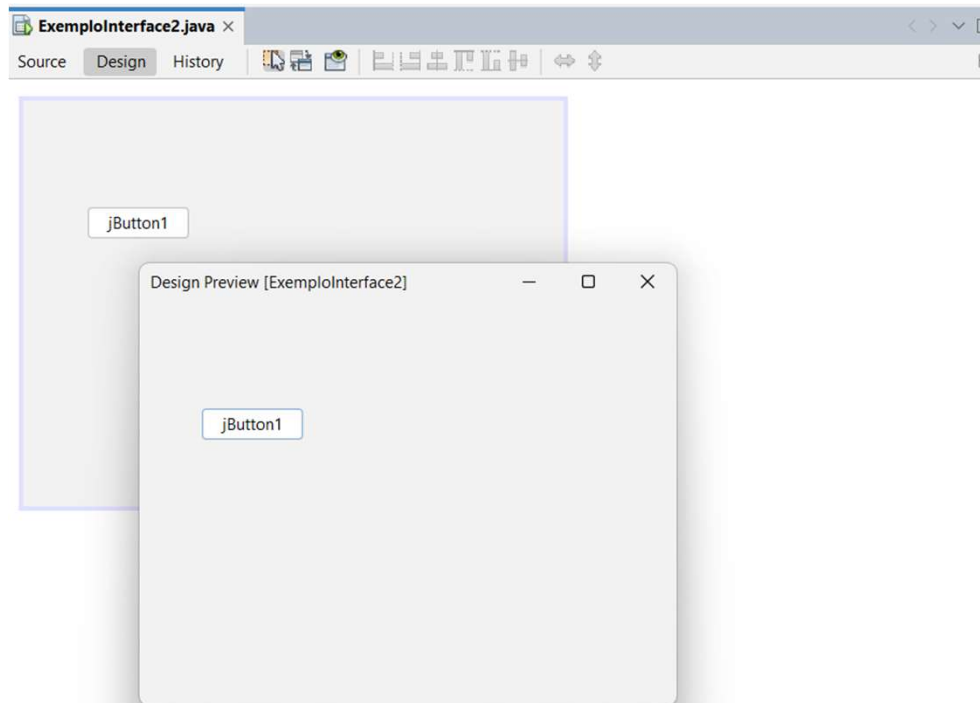
Java Swing



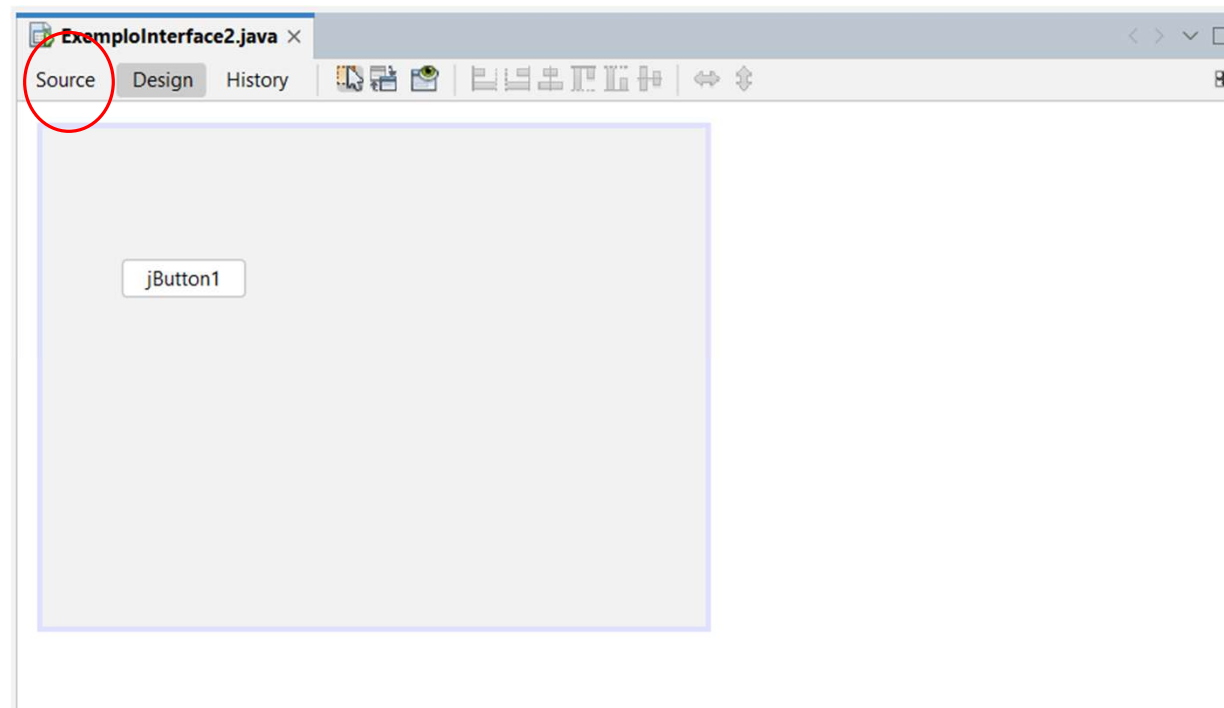
Java Swing



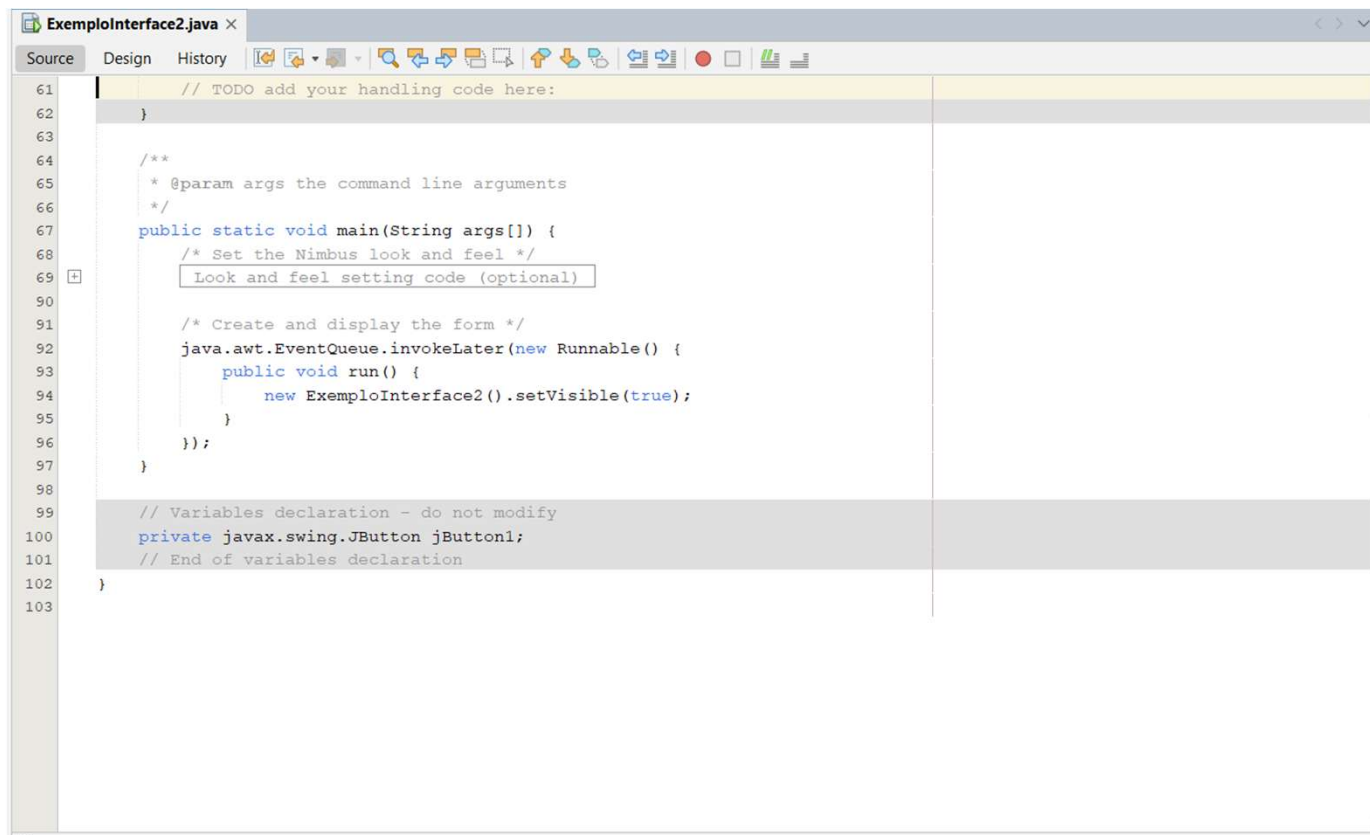
Java Swing



Java Swing



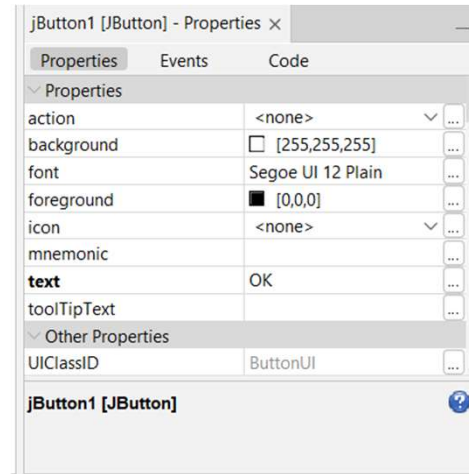
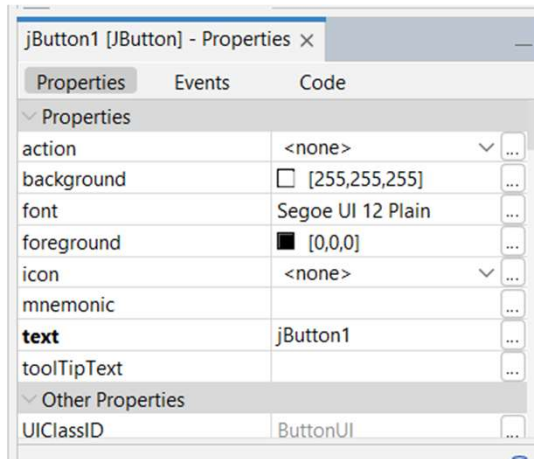
Java Swing



```
61 // TODO add your handling code here:
62 }
63
64 /**
65  * @param args the command line arguments
66  */
67 public static void main(String args[]) {
68     /* Set the Nimbus look and feel */
69     Look and feel setting code (optional)
70
71     /* Create and display the form */
72     java.awt.EventQueue.invokeLater(new Runnable() {
73         public void run() {
74             new ExemploInterface2().setVisible(true);
75         }
76     });
77 }
78
79 // Variables declaration - do not modify
80 private javax.swing.JButton jButton1;
81 // End of variables declaration
82 }
83
```

Java Swing

- Agora, procure por “text” nas propriedades do jButton e altere para OK



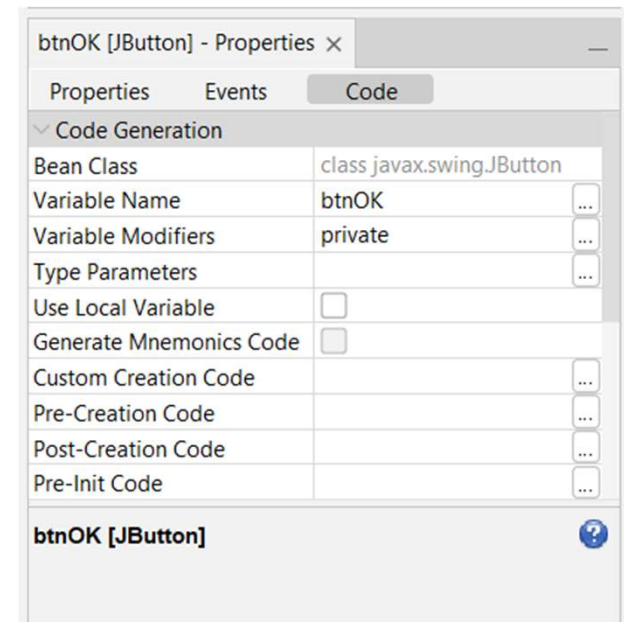
Java Swing

- Ele abrira novamente o código e automaticamente ele irá criar um método chamado “actionPerformed”
- Este método será executado toda vez que o botão for pressionado...

```
43 private void initialize() {  
44     frame = new JFrame();  
45     frame.setBounds(100, 100, 450, 300);  
46     frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
47  
48     JPanel panel = new JPanel();  
49     frame.getContentPane().add(panel, BorderLayout.NORTH);  
50  
51     JButton btnOK = new JButton("OK");  
52     btnOK.addActionListener(new ActionListener() {  
53         public void actionPerformed(ActionEvent e) {  
54             JOptionPane.showMessageDialog(frame, "Oi mundo!");  
55         }  
56     });  
57     frame.getContentPane().add(btnOK, BorderLayout.CENTER);  
58 }  
59  
60  
61  
62 }
```

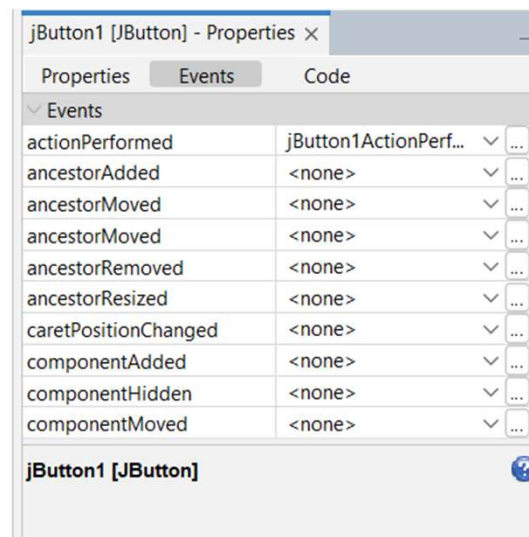
Java Swing

- Além disso, existem mais duas abas de cada componente
 - O terceiro é “Code”, onde é possível verificar e alterar o nome do componente e seguir a convenção
 - Altere o nome para “btnOK”
- Nesta aba também é possível alterar a visibilidade do botão, parâmetros nas chamadas de métodos entre outras coisas...



Java Swing

- Além disso, existem mais duas abas de cada componente
- A segunda é “Events”, onde é possível programar qualquer tipo de evento para cada componente

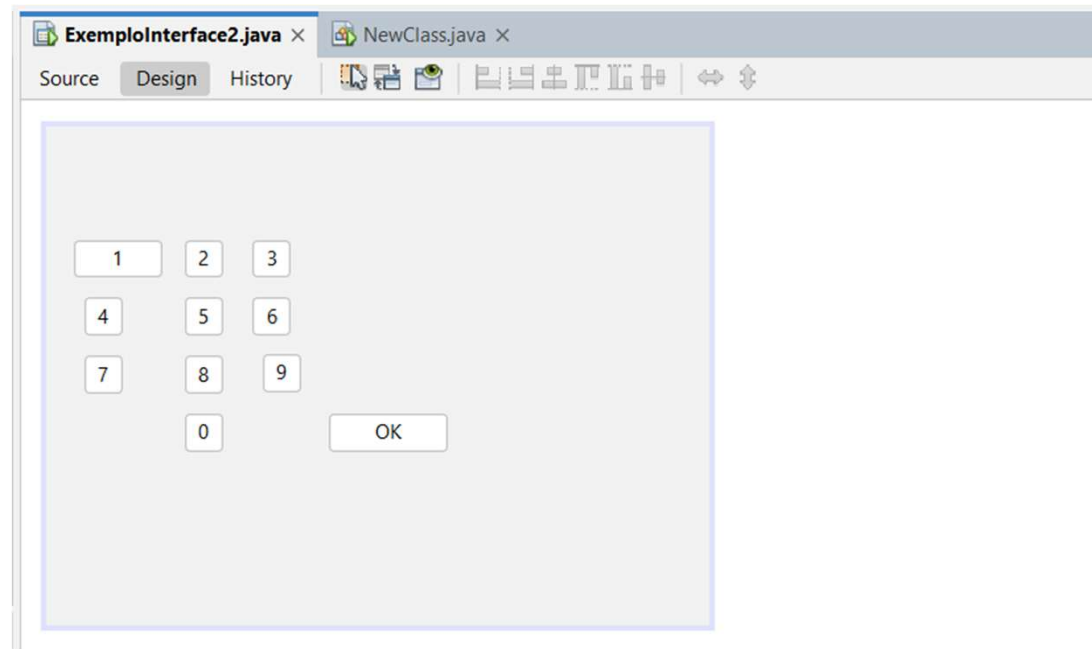


Java Swing

- Agora, dê dois cliques no botão e adicione o seguinte código:

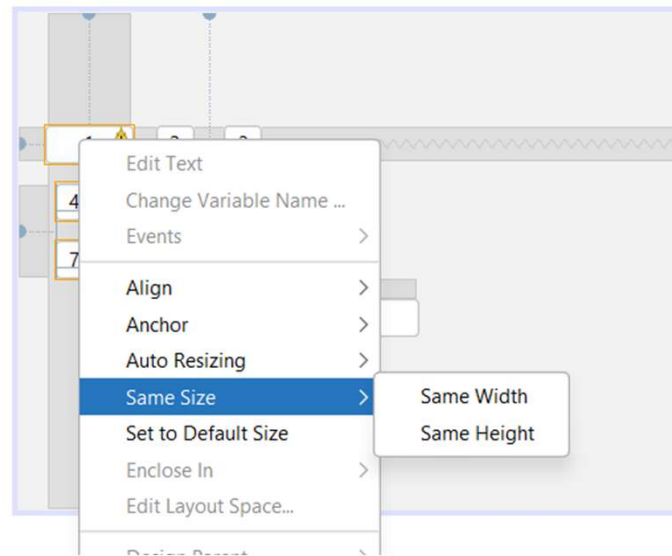
```
13 public class ExemploInterface2 extends javax.swing.JFrame {
14
15     /**
16      * Creates new form ExemploInterface2
17      */
18     public ExemploInterface2() {
19         initComponents();
20     }
21
22     /**
23      * This method is called from within the constructor to initialize the form.
24      * WARNING: Do NOT modify this code. The content of this method is always
25      * regenerated by the Form Editor.
26      */
27     @SuppressWarnings("unchecked")
28     Generated Code
61
62     private void btnOKActionPerformed(java.awt.event.ActionEvent evt) {
63         JOptionPane.showMessageDialog(parentComponent: null, message: "Oi mundo!");
64     }
65
```

Java Swing – Calculadora: monta esse tela



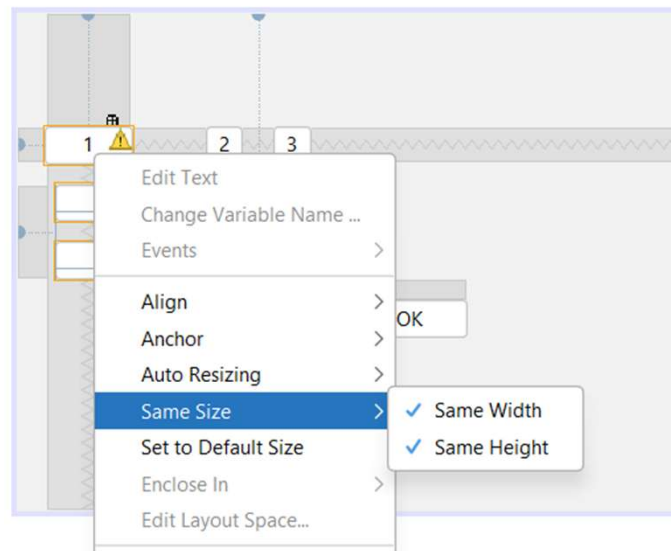
Java Swing

- Para deixar alguns componentes com o mesmo tamanho, selecione os itens apertando a tecla “Ctrl” e selecione por último a que você quer copiar o tamanho:



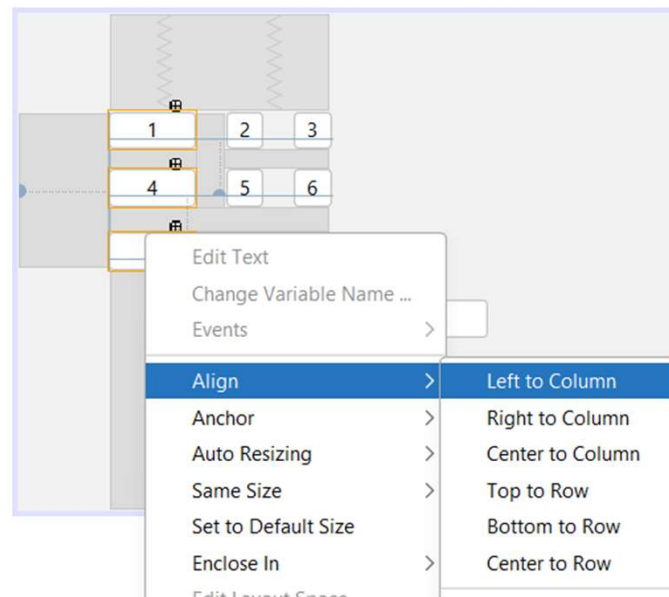
Java Swing

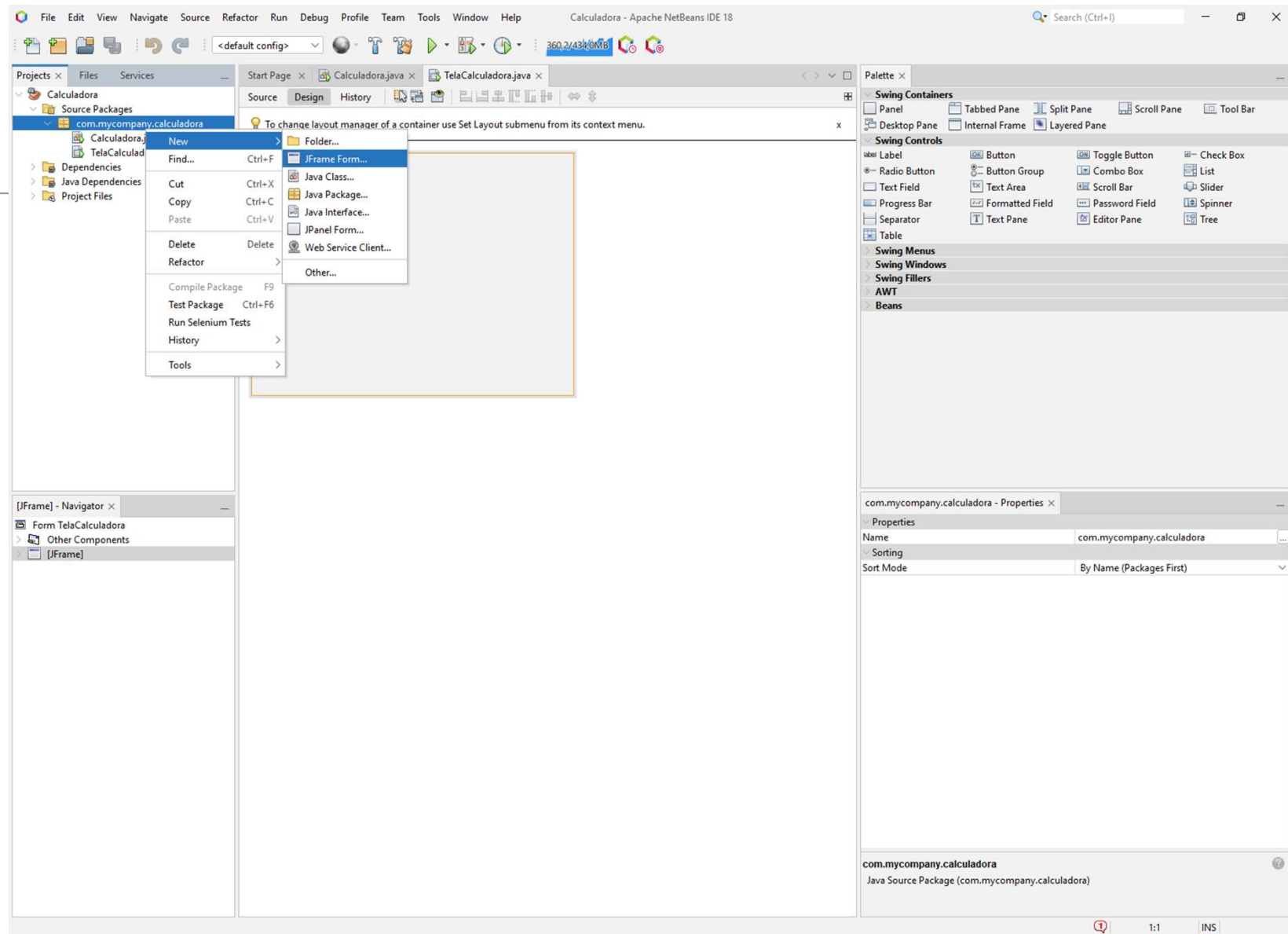
- Para deixar alguns componentes com o mesmo tamanho, selecione os itens apertando a tecla “Ctrl” e selecione por último a que você quer copiar o tamanho:



Java Swing

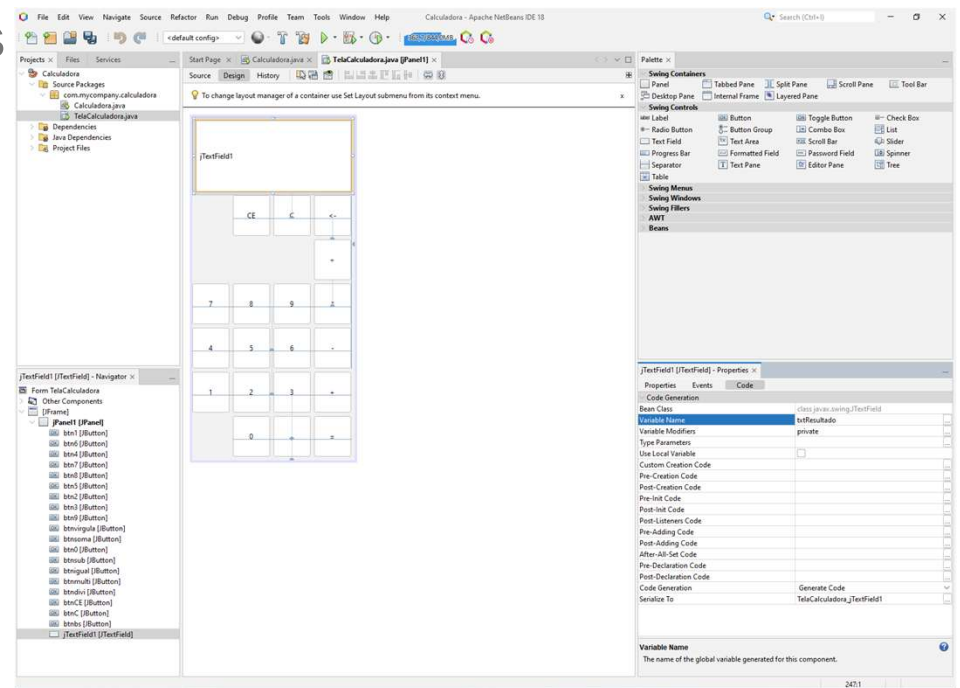
- Agora, para alinhar, faça o mesmo, selecione os componentes deixando por último o elemento o qual você quer deixar igual:



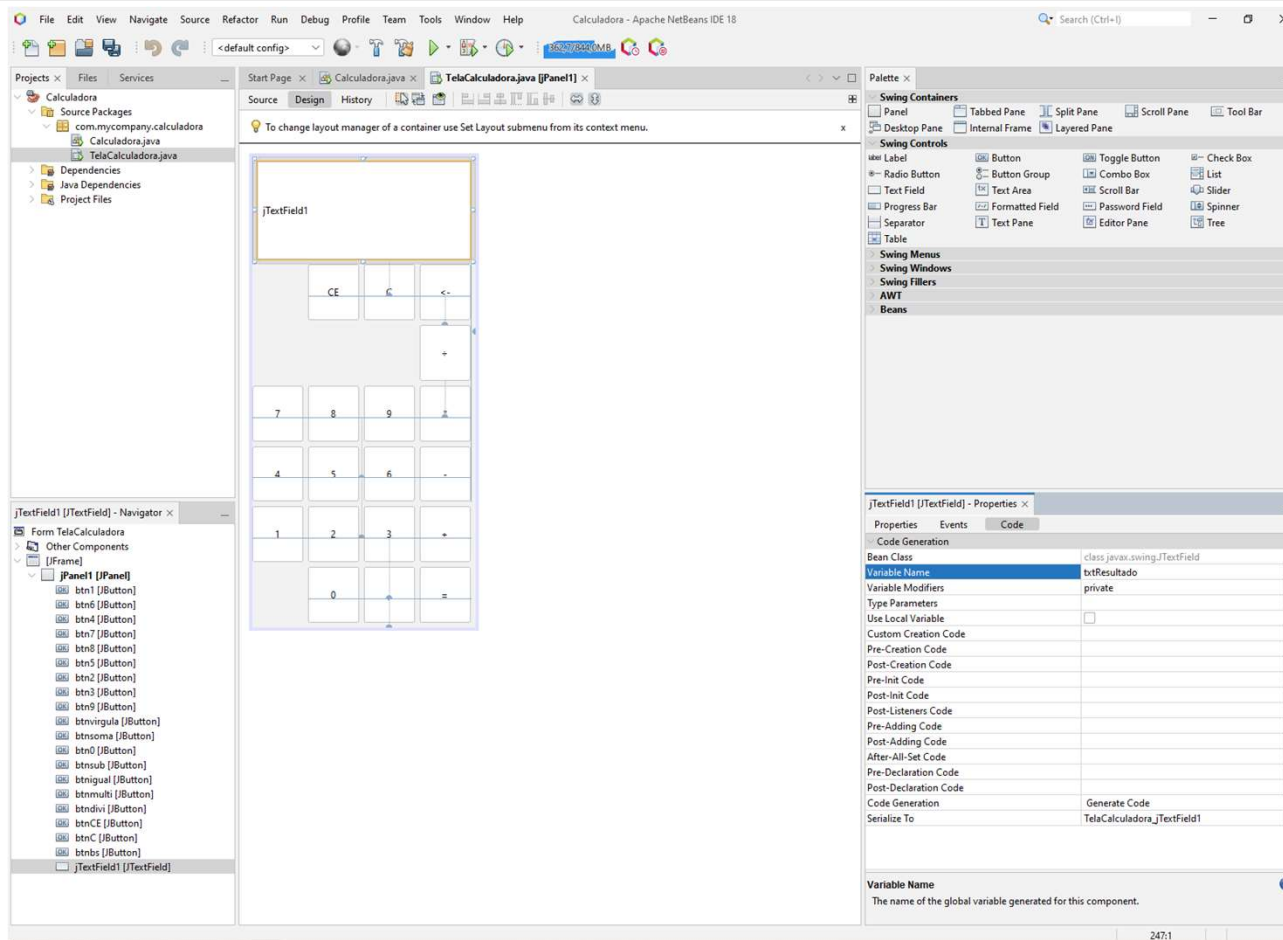


Desenvolvimento de uma calculadora

- Adicione os botões da calcular, coloque o nome conforme convenção (btnNOME)
- Coloque só as principais operações
- Adicione o Textfield



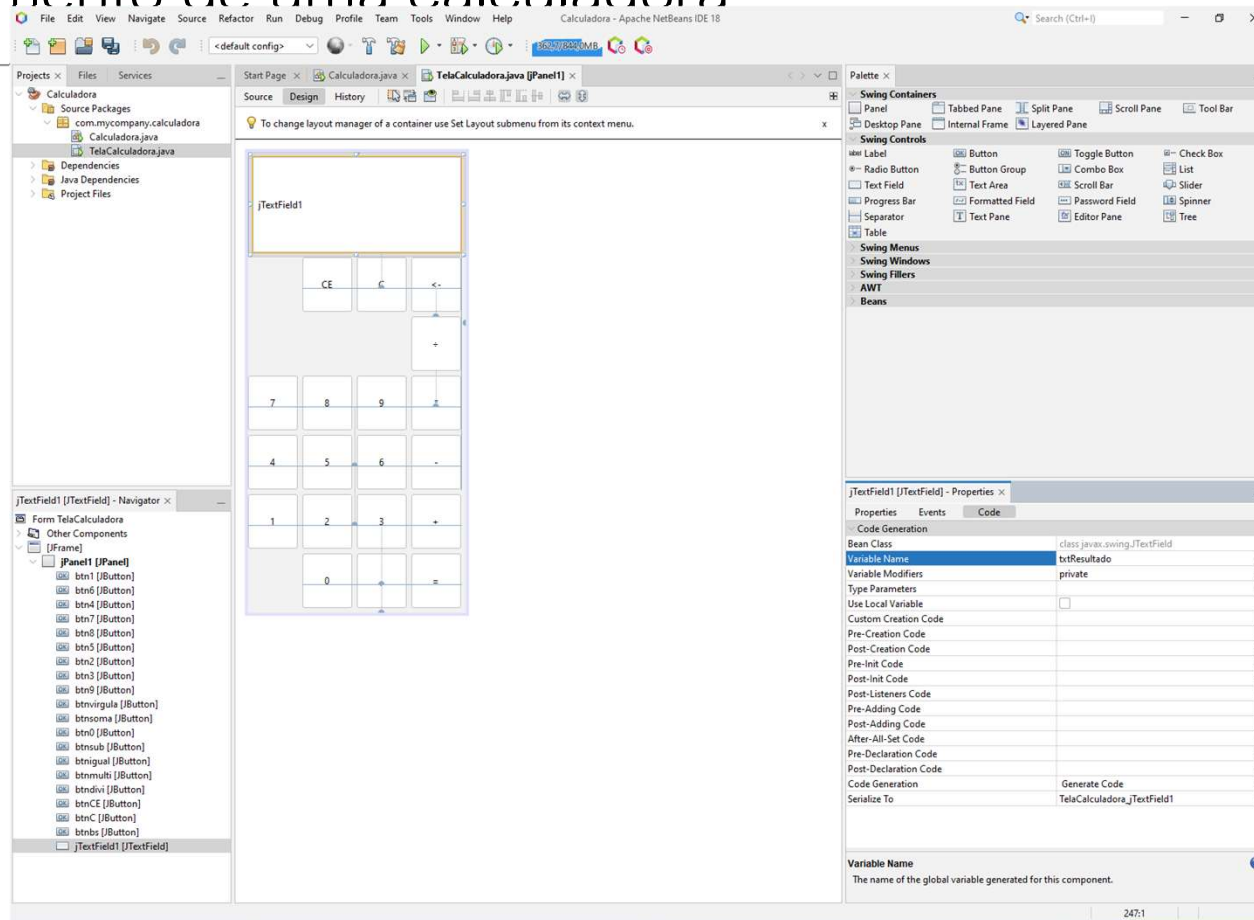
Desenvolvimento de uma calculadora



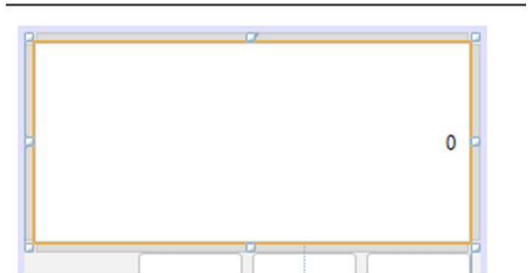
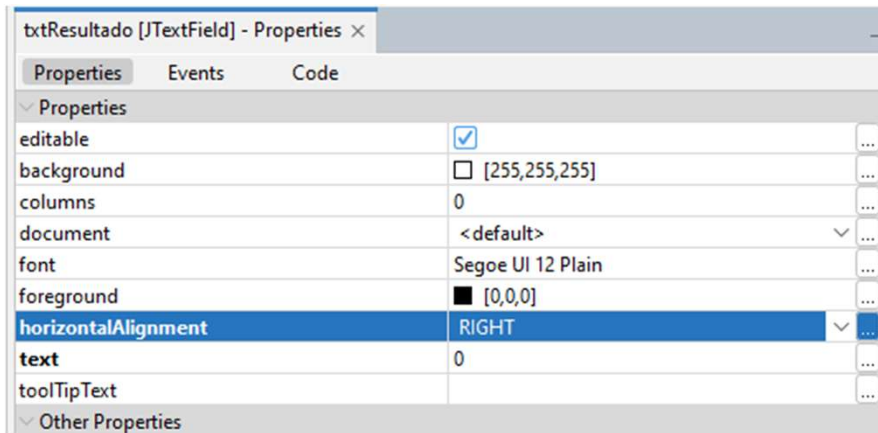
Adicione os outros botões....

- Dificuldades em manter os botões na posição?
 - Adicione um Panel e coloque os botões dentro, isso vai ajudar...

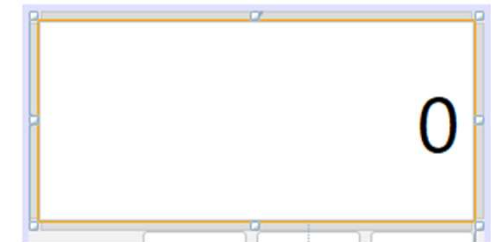
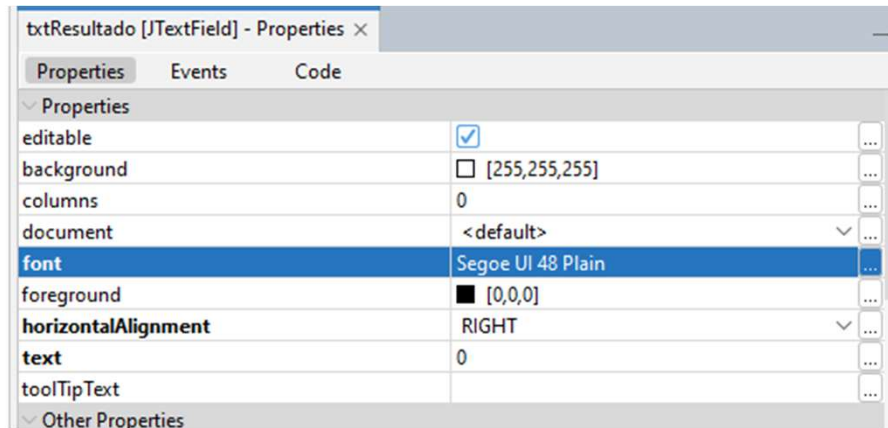
Desenvolvimento de uma calculadora



Desenvolvimento de uma calculadora



Desenvolvimento de uma calculadora



Agora, precisamos configurar as ações dos botões

- No evento de ação (do clique) do botão 0 (btn0) adicionei a chamada da função setText do TextBox txtResultado:

```
256 private void btn0ActionPerformed(java.awt.event.ActionEvent evt) {  
258     txtResultado.setText("0");  
259 }  
---
```

- Podemos fazer isso nos outros botões:

```
private void btn1ActionPerformed(java.awt.event.ActionEvent evt) {  
277     txtResultado.setText("1");  
278 }  
279  
private void btn2ActionPerformed(java.awt.event.ActionEvent evt) {  
281     txtResultado.setText("2");  
282 }  
283
```

Desenvolvendo uma calculadora

- Agora teste o clique no botão...
 - O que acontece quando você aperta um botão e depois o outro?

Desenvolvendo uma calculadora

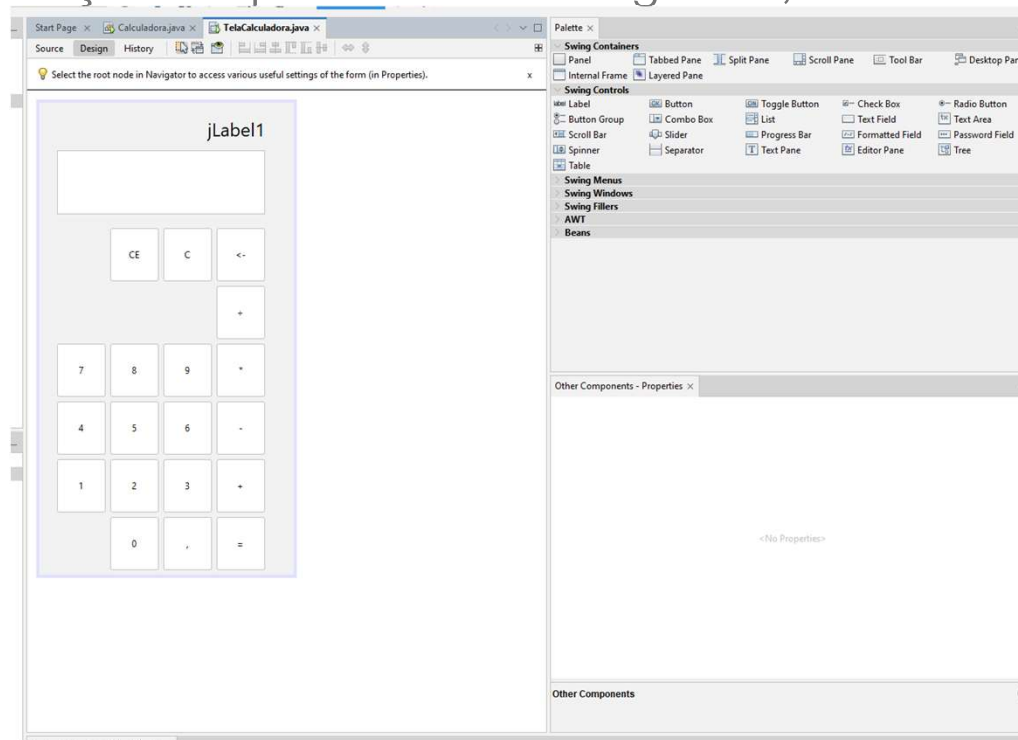
- Agora teste o clique no botão...
 - O que acontece quando você aperta um botão e depois o outro?
 - Sobrescreve o valor, certo?
- Para isso, precisamos melhorar nosso código..

Desenvolvendo uma calculadora

```
273 private void btn0ActionPerformed(java.awt.event.ActionEvent evt) {  
274     txtResultado.setText(txtResultado.getText()+"0");  
275 }  
276  
277 private void btn1ActionPerformed(java.awt.event.ActionEvent evt) {  
278     txtResultado.setText(txtResultado.getText()+"1");  
279 }  
280  
281 private void btn2ActionPerformed(java.awt.event.ActionEvent evt) {  
282     txtResultado.setText(txtResultado.getText()+"2");  
283 }  
284  
285 private void btn3ActionPerformed(java.awt.event.ActionEvent evt) {  
286     txtResultado.setText(txtResultado.getText()+"3");  
287 }
```

Desenvolvendo uma calculadora

- Agora, precisamos implementar as operações, mas antes, para lembrarmos e termos a informação do primeiro valor digitado, vamos adicionar um label



Desenvolvendo uma calculadora

- Renomeei um label para lblValor1, e fui pro actionPerformed da soma (dei dois cliques no botão +)

```
331  
332  private void btnsomaActionPerformed(java.awt.event.ActionEvent evt) {  
333      lblValor1.setText(txtResultado.getText());  
334  }
```



Desenvolvendo uma calculadora

- Mas ainda, precisamos limpar o texto do txtResultado

```
333  
334  private void btnsomaActionPerformed(java.awt.event.ActionEvent evt) {  
335      lblValor1.setText(text: txtResultado.getText());  
336      txtResultado.setText(text: "");  
337  }  
338
```

Desenvolvendo uma calculadora



Desenvolvendo uma calculadora

- E ainda vamos precisar de algumas variáveis novas
- Declaremos como globais as variáveis valor1 e valor2 do tipo double e
- Uma String operacao para armazenar a operação que será efetuada

```
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132  
133  
134  
135  
136  
137  
138  
139  
140  
141  
142  
143  
144  
145  
146  
147  
148  
149  
150  
151  
152  
153  
154  
155  
156  
157  
158  
159  
160  
161  
162  
163  
164  
165  
166  
167  
168  
169  
170  
171  
172  
173  
174  
175  
176  
177  
178  
179  
180  
181  
182  
183  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000
```

Desenvolvendo uma calculadora

- E agora, no ActionPerformed do botão soma, vamos adicionar as atribuições das variáveis criadas:

```
335 private void btnsomaActionPerformed(java.awt.event.ActionEvent evt) {  
336     lblValor1.setText(text: txtResultado.getText());  
337     txtResultado.setText(text: "");  
338     valor1 = Double.parseDouble(text: lblValor1.getText());  
339     operacao = "soma";  
340  
341 }
```

Desenvolvendo uma calculadora

- E agora, precisamos manipular no botão “=” para calcular e apresentar o resultado...

```
357 private void btnigualActionPerformed(java.awt.event.ActionEvent evt) {  
358     double resultado;  
359     valor2 = Double.parseDouble(txtResultado.getText());  
360     if(operacao=="soma"){  
361         resultado = valor1 + valor2;  
362         lblValor1.setText (valor1+"+"+valor2+"=");  
363         txtResultado.setText(String.valueOf(resultado));  
364     }  
}
```


Desenvolvendo uma calculadora

```
private void btnigualActionPerformed(java.awt.event.ActionEvent evt) {  
    double resultado;  
    valor2 = Double.parseDouble(txtResultado.getText());  
    if(operacao=="soma"){  
        resultado = valor1 + valor2;  
        lblValor1.setText (valor1+"+"+valor2+"=");  
        txtResultado.setText(String.valueOf(resultado));  
    }  
    else if(operacao=="sub"){  
        resultado = valor1 - valor2;  
        lblValor1.setText (valor1+"-"+valor2+"=");  
        txtResultado.setText(String.valueOf(resultado));  
    }  
    else if(operacao=="multi"){  
        resultado = valor1 * valor2;  
        lblValor1.setText (valor1+"x"+valor2+"=");  
        txtResultado.setText(String.valueOf(resultado));  
    }  
    else if(operacao=="div"){  
        resultado = valor1 / valor2;  
        lblValor1.setText (valor1+"÷"+valor2+"=");  
        txtResultado.setText(String.valueOf(resultado));  
    }  
}
```

Desenvolvendo uma calculadora

```
396 private void btnsubActionPerformed(java.awt.event.ActionEvent evt) {  
397     lblValor1.setText(text: txtResultado.getText());  
398     txtResultado.setText(t: "");  
399     valor1 = Double.parseDouble(s: lblValor1.getText());  
400     operacao = "sub";  
401 }
```

```
388 private void btnmultiActionPerformed(java.awt.event.ActionEvent evt) {  
389     lblValor1.setText(text: txtResultado.getText());  
390     txtResultado.setText(t: "");  
391     valor1 = Double.parseDouble(s: lblValor1.getText());  
392     operacao = "multi"; // TODO add your handling code here:  
393 }  
394
```

```
306 private void btndiviActionPerformed(java.awt.event.ActionEvent evt) {  
307     lblValor1.setText(text: txtResultado.getText());  
308     txtResultado.setText(t: "");  
309     valor1 = Double.parseDouble(s: lblValor1.getText());  
310     operacao = "div";  
311 }
```

Desenvolvendo uma calculadora

- E o C e o CE?
- E as outras operações?
 - Agora, é com vocês, façam conforme a curiosidade e a vontade de vocês..

Java Swing – Outros componentes

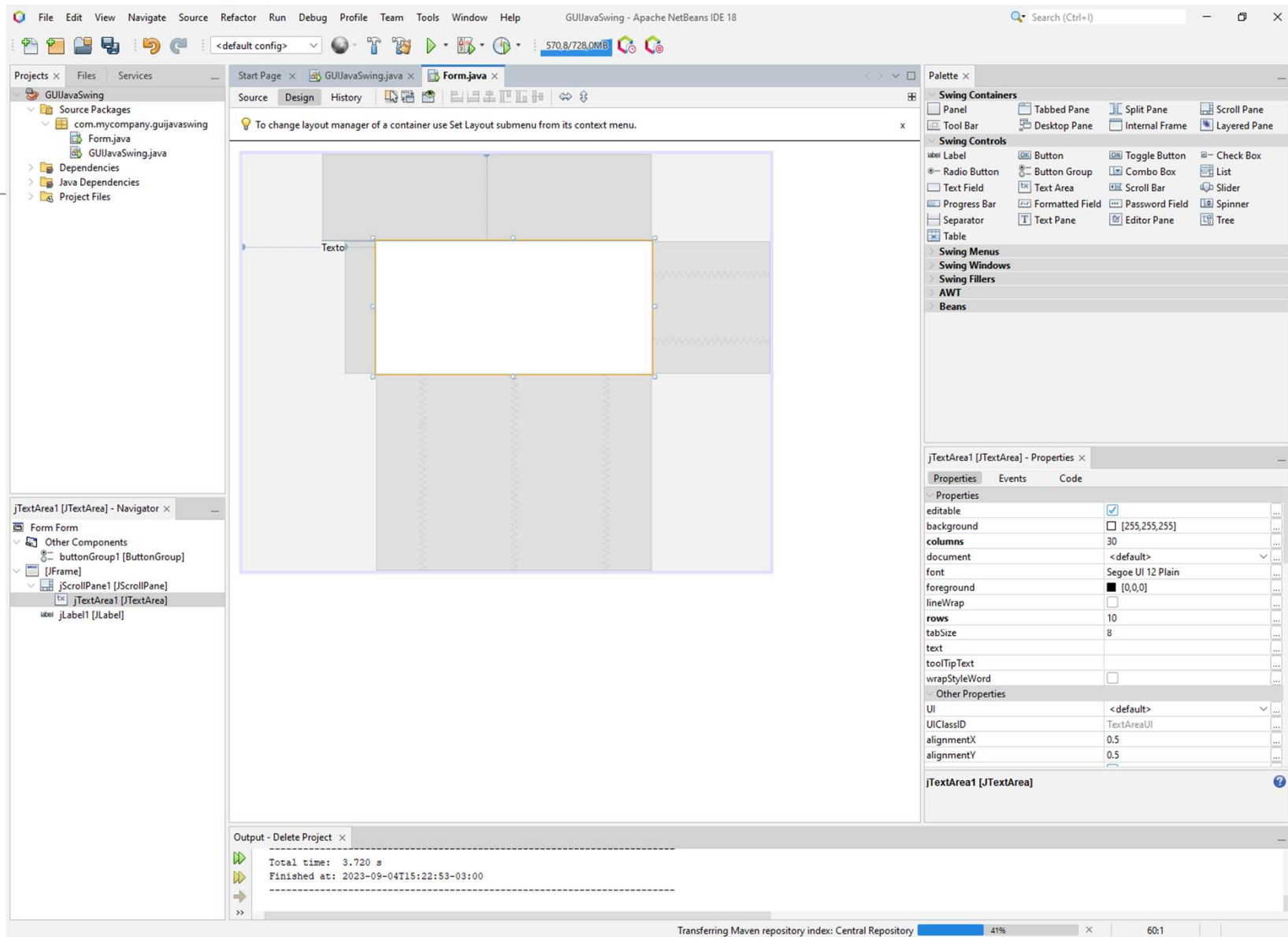


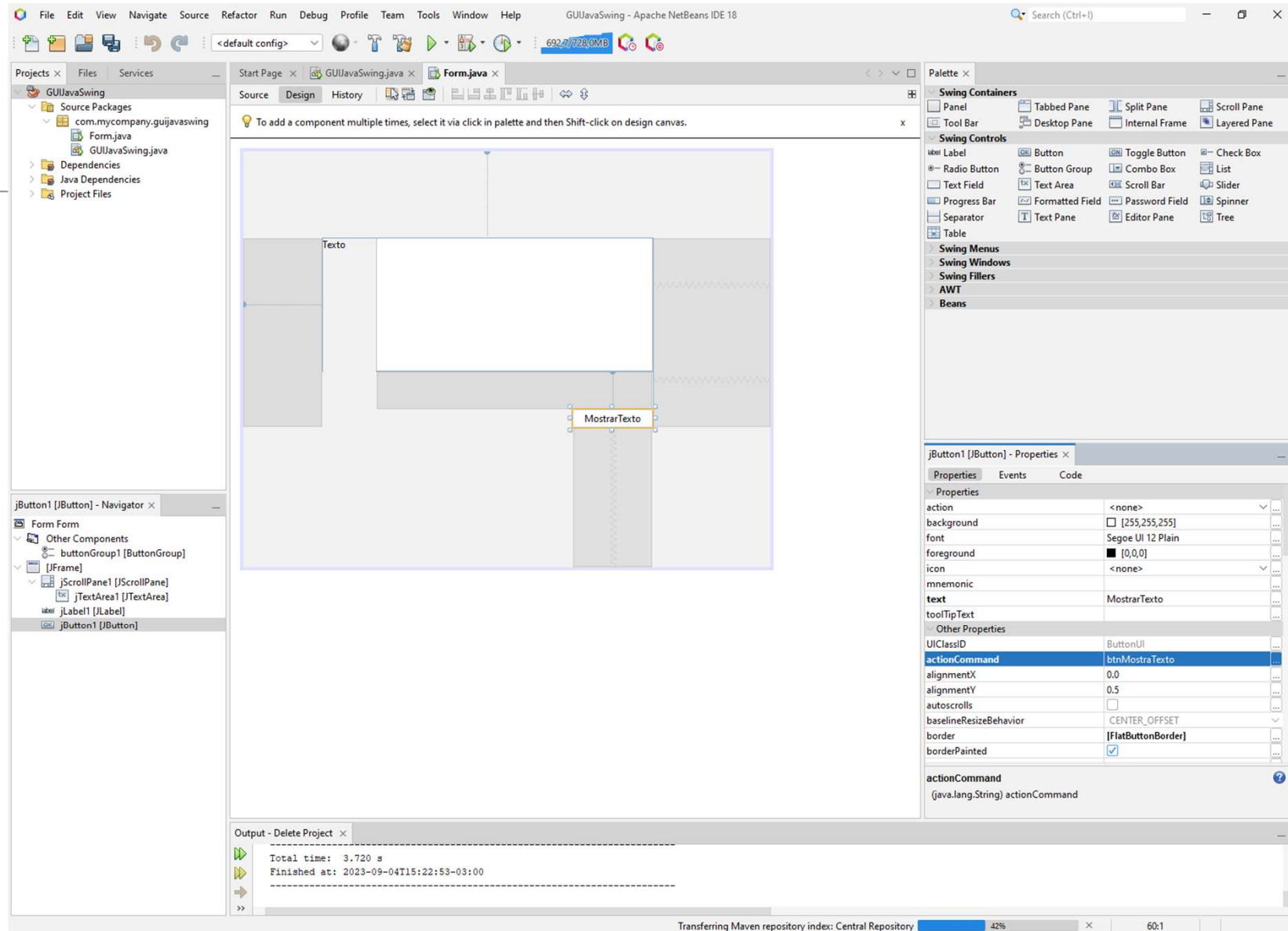
Java Swing

- Na aula de hoje, vamos trabalhar com outros componentes:
 - Text Area
 - Radio Button
 - CheckBox
 - ComboBox
 - Table

Text Area

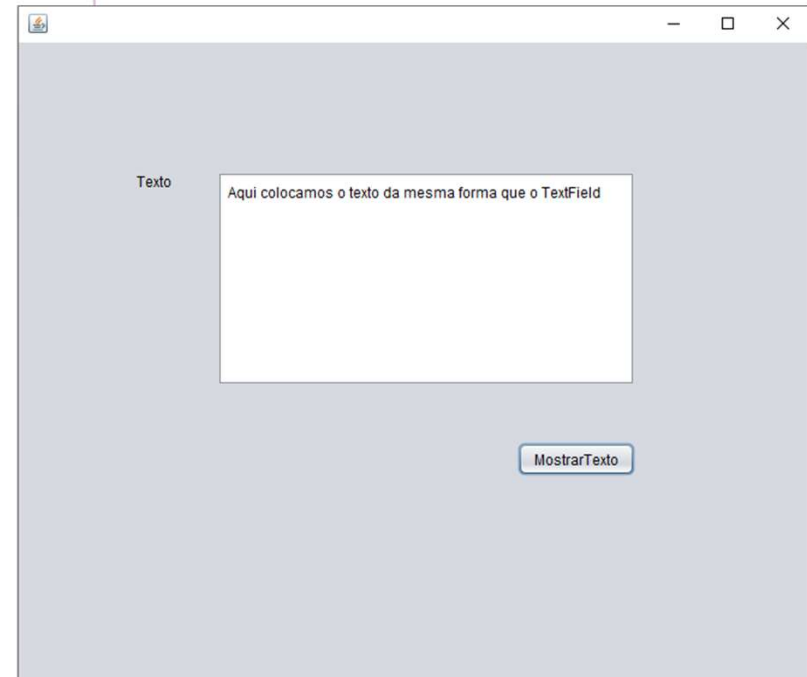
- Adicione um JFrame Form
- E dentro dele, adicione um label e um text area





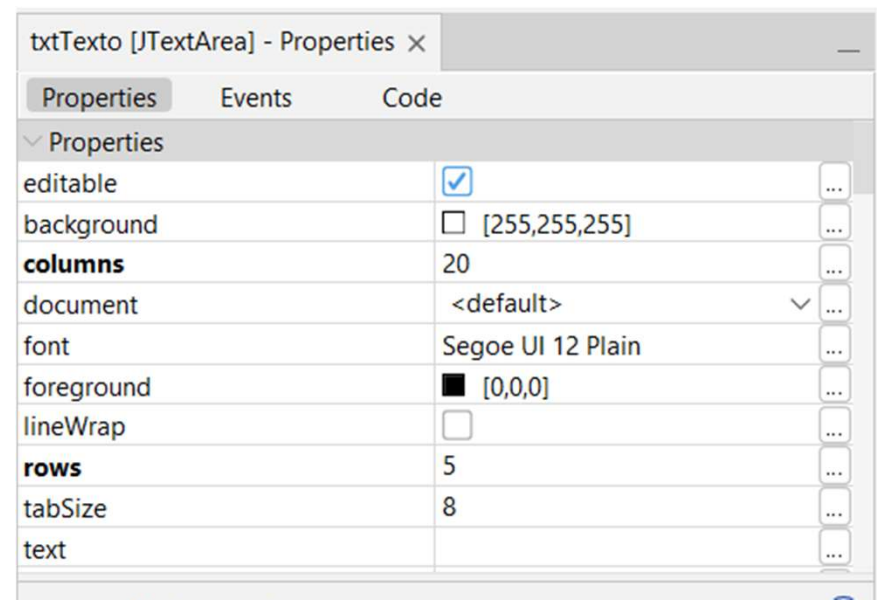
Text Area

```
private void btnMostraTextoActionPerformed(java.awt.event.ActionEvent evt) {  
    txaTexto.setText("Aqui colocamos o texto da mesma forma que o TextField");  
}
```



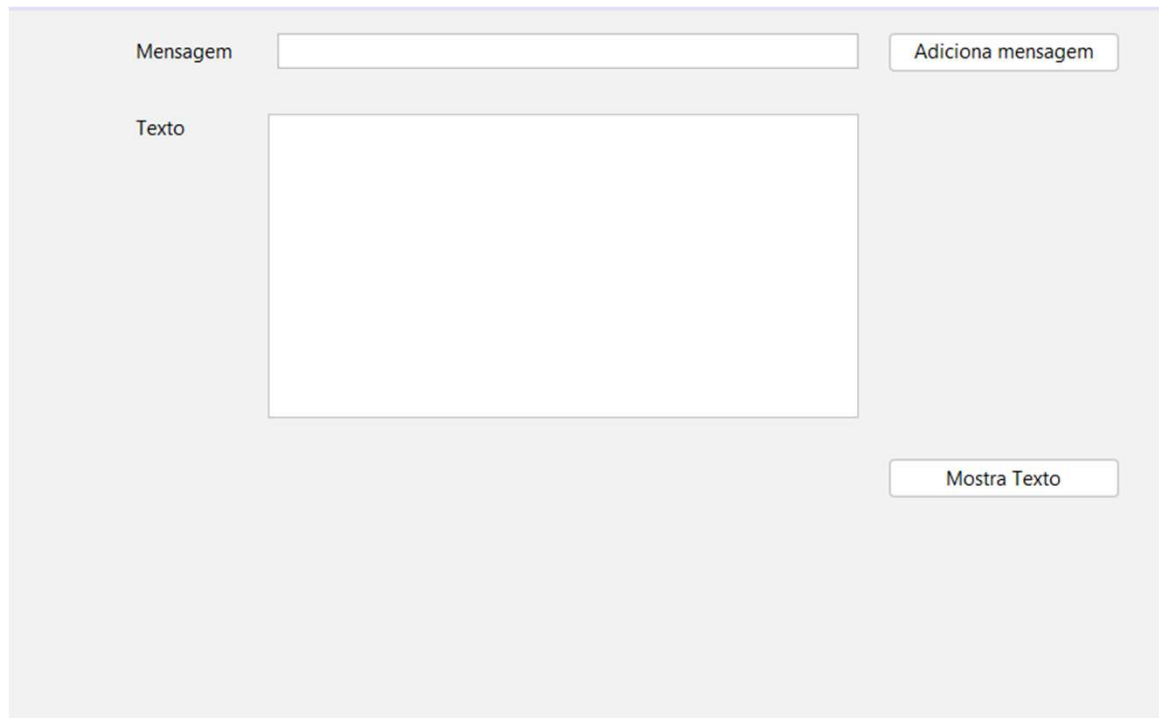
Text Area

- A diferença do text field é que aqui conseguimos mudar a quantidade de colunas e linhas e ir adicionando textos dentro dele.



Text Area

- Agora adicione um text field e outro botão



The image shows a web form with a light gray background. It contains the following elements:

- A label "Mensagem" followed by a single-line text input field.
- A button labeled "Adiciona mensagem" located to the right of the "Mensagem" input field.
- A label "Texto" followed by a large multi-line text area.
- A button labeled "Mostra Texto" located at the bottom right of the form.

Text Area

- Adicione o código abaixo na ação do botão adicionado

```
private void btnAddMsgActionPerformed(java.awt.event.ActionEvent evt) {  
    txaTexto.append(str:txtMensagem.getText());  
}
```

Text Area

- E como ficou? Para melhorar, podemos limpar o textfield e quebrar a linha após cada mensagem adicionada.

```
private void btnAddMsgActionPerformed(java.awt.event.ActionEvent evt) {  
    txaTexto.append(txtMensagem.getText()+"\n");  
    txtMensagem.setText("");  
}
```

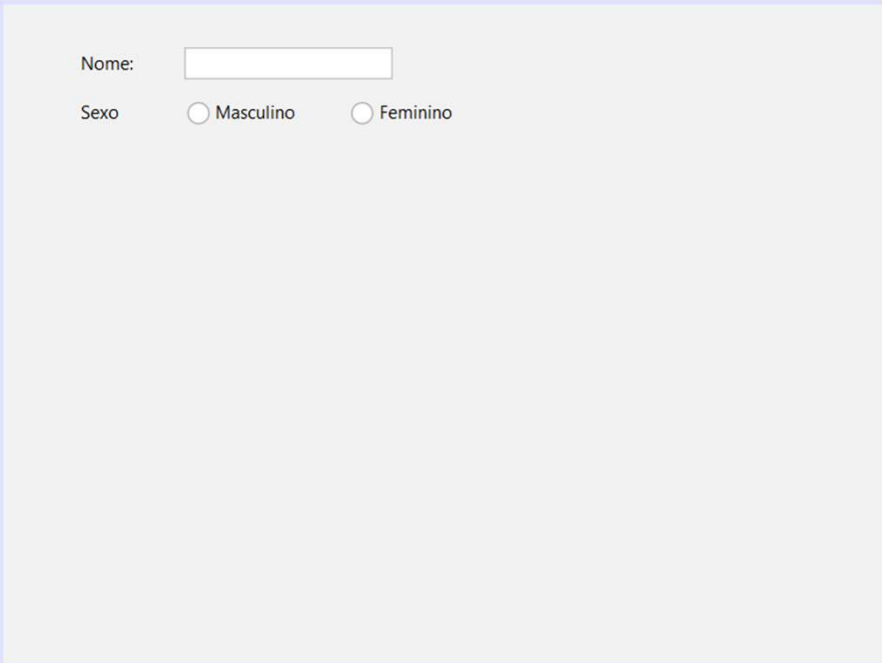
- Entenderam para que funciona o método append?

Radion Button

- É usado para permitir que os usuários escolham uma única opção de um conjunto pré-definido de opções mutuamente exclusivas.
- Eles são chamados de "radio buttons" porque se assemelham aos botões de rádio físicos em um rádio antigo, onde você só pode selecionar uma estação de rádio por vez.

Radio Button

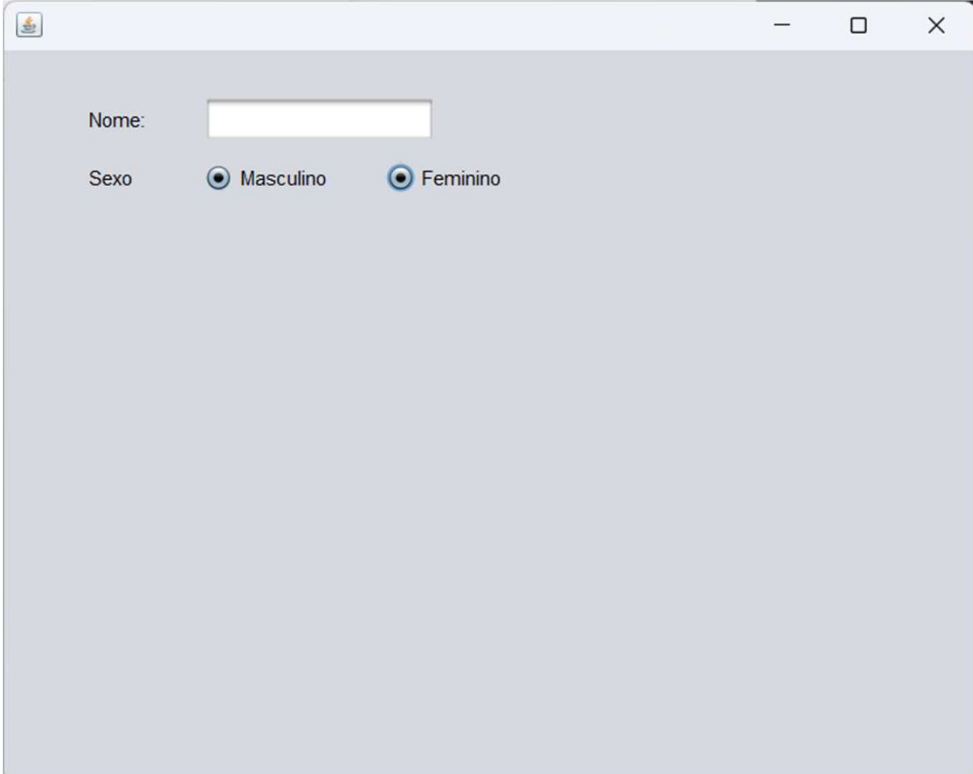
- Adicione um novo JForm e adicione um text field dois Radion Button e compile o código..



The image shows a Java Swing window titled "Form" with a light gray background. Inside the window, there are two labels: "Nome:" and "Sexo:". The "Nome:" label is positioned to the left of a white text field with a thin gray border. The "Sexo:" label is positioned to the left of two radio buttons. The first radio button is labeled "Masculino" and the second is labeled "Feminino". Both radio buttons are currently unselected.

Radio Button

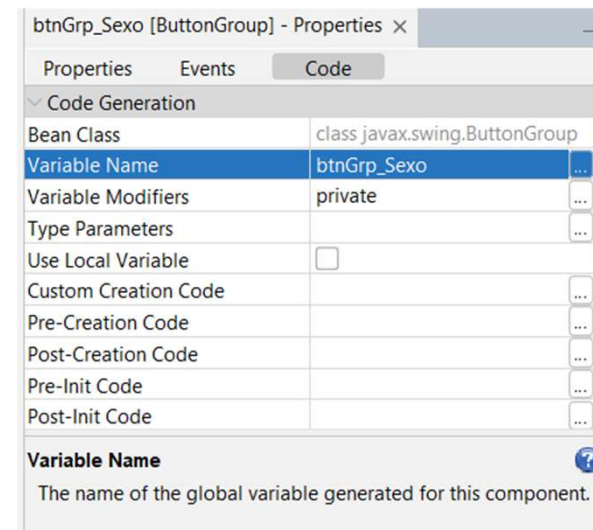
- Funcionou? Este botão não deve possibilitar selecionar os dois
- Como resolver?



The image shows a web form within a browser window. The form has a light gray background. At the top, there is a label 'Nome:' followed by a white text input field. Below this, there is a label 'Sexo' followed by two radio buttons. The first radio button is selected and is labeled 'Masculino'. The second radio button is also selected and is labeled 'Feminino'. The browser window has a standard title bar with a small icon on the left and minus, maximize, and close buttons on the right.

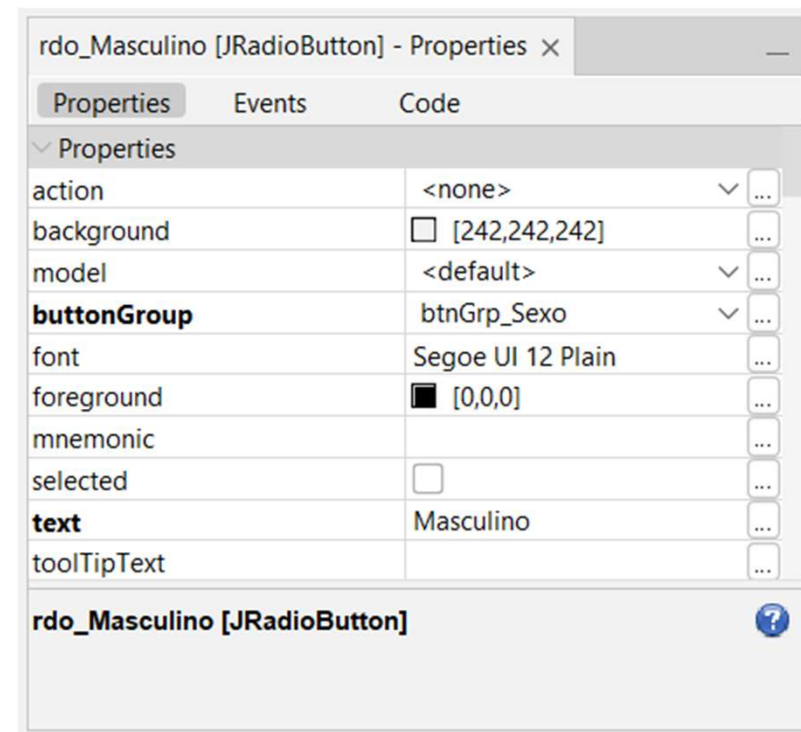
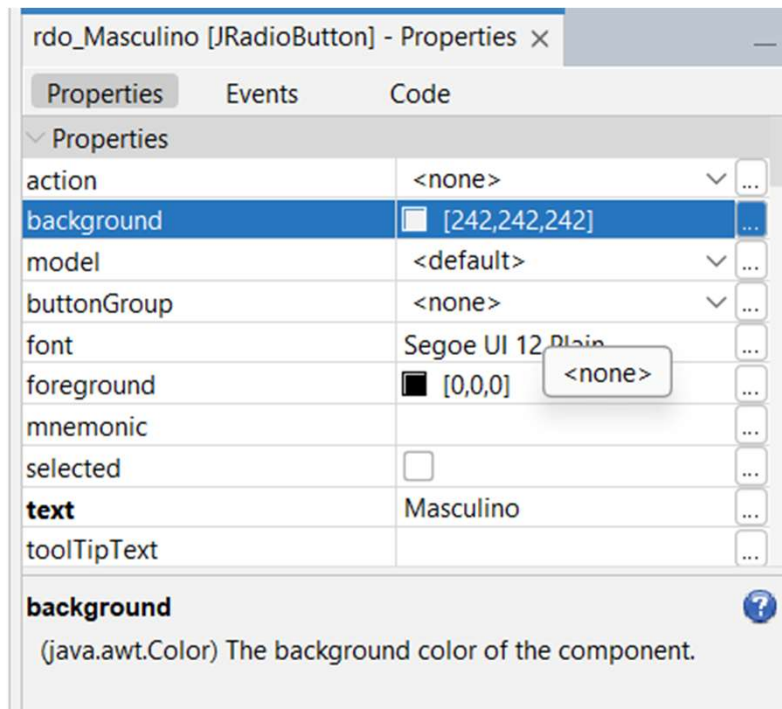
Radio Button

- Adicione um Button Group
- Ele não aparecerá no formulário



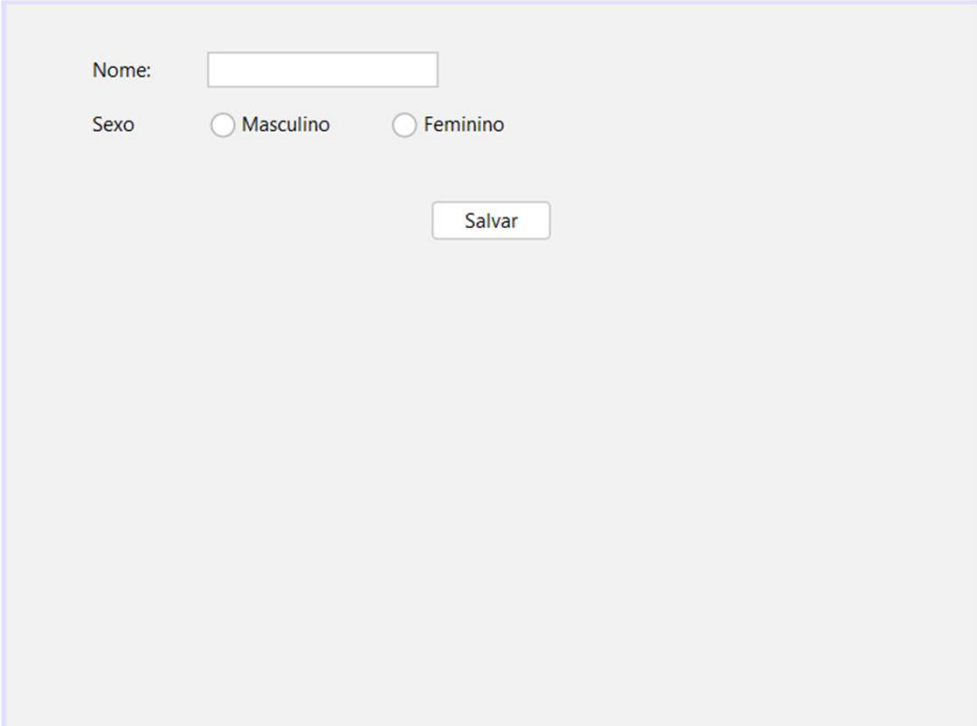
Button Group

- Selecione o RadioButton e vá em propriedades



Radio Button

- E como verificar qual está selecionado?
- Adicione um botão Salvar



Nome:

Sexo ☐ Masculino ☐ Feminino

Radio Button

- Declare uma variável global do tipo char chamada sexo
- Na ação do botão Salvar, adicione o código abaixo:

```
if(rdo_Masculino.isSelected()){  
    sexo = 'M';  
    JOptionPane.showMessageDialog(parentComponent: null,  
        "Sexo selecionado: "+sexo, title: "Alerta!" , messageType: JOptionPane.OK_OPTION);  
}  
else if(rdo_Feminino.isSelected()){  
    sexo = 'F';  
    JOptionPane.showMessageDialog(parentComponent: null,  
        "Sexo selecionado: "+sexo, title: "Alerta!" , messageType: JOptionPane.OK_OPTION);  
}  
else{  
    JOptionPane.showMessageDialog(parentComponent: null,  
        message: "Selecione um sexo", title: "Erro!" , messageType: JOptionPane.ERROR_MESSAGE);  
}
```

Radio Button

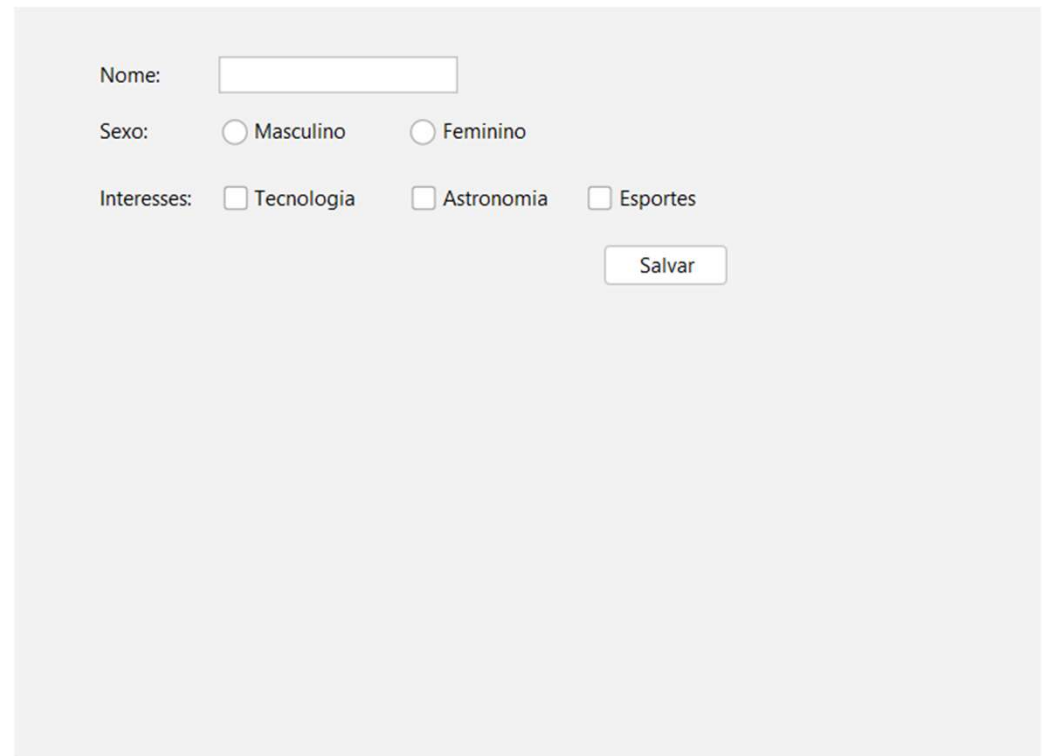
```
if(rdo_Masculino.isSelected()){
    sexo = 'M';
    JOptionPane.showMessageDialog(parentComponent: null,
        "Sexo selecionado: "+sexo, title: "Alerta!" , messageType: JOptionPane.OK_OPTION);
}
else if(rdo_Feminino.isSelected()){
    sexo = 'F';
    JOptionPane.showMessageDialog(parentComponent: null,
        "Sexo selecionado: "+sexo, title: "Alerta!" , messageType: JOptionPane.OK_OPTION);
}
else{
    JOptionPane.showMessageDialog(parentComponent: null,
        message: "Selecione um sexo", title: "Erro!" , messageType: JOptionPane.ERROR_MESSAGE);
}
```

Check Box

- Agora, vamos trabalhar com Check Box
- Permite aos usuários selecionar ou desmarcar uma ou mais opções de um conjunto de opções independentes.
- Os checkboxes são frequentemente usados quando você deseja que os usuários tenham a capacidade de escolher várias opções dentre várias disponíveis.

Check Box

- Adicione estes check box conforme a imagem, renomeie conforme o padrão visto em aula
- `chk_Tecnologia`, por exemplo



Nome:

Sexo: ☐ Masculino ☐ Feminino

Interesses: ☐ Tecnologia ☐ Astronomia ☐ Esportes

Check Box

- Declare três atributos do tipo bool

```
char sexo;  
boolean tecnologia, astronomia, esportes;
```

- E na ação do botão:

```
if(chk_Tecnologia.isSelected()){  
    tecnologia = true;  
}  
else{  
    tecnologia = false;  
}  
if(chk_Astronomia.isSelected()){  
    astronomia = true;  
}  
else{  
    astronomia = false;  
}  
if(chk_Esportes.isSelected()){  
    esportes = true;  
}  
else{  
    esportes = false;  
}  
JOptionPane.showMessageDialog(parentComponent: null, "Tecnologia: "+tecnologia+"\nAstronomia: "+astronomia+  
    "\nEsportes: "+esportes, title: "Alerta", messageType: JOptionPane.INFORMATION_MESSAGE);
```



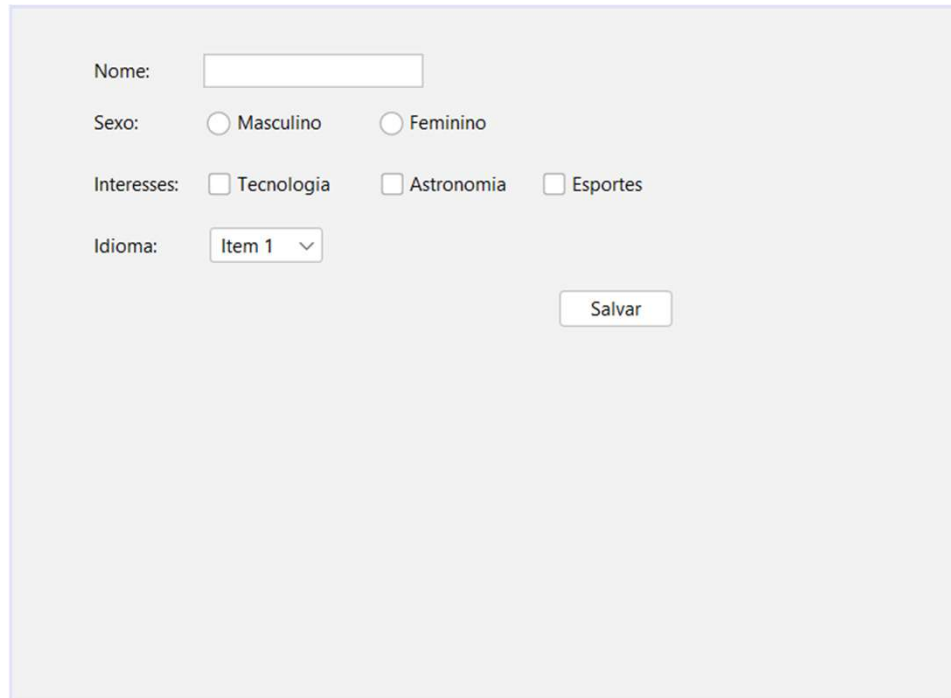
```
if(chk_Tecnologia.isSelected()){
    tecnologia = true;
}
else{
    tecnologia = false;
}
if(chk_Astronomia.isSelected()){
    astronomia = true;
}
else{
    astronomia = false;
}
if(chk_Esportes.isSelected()){
    esportes = true;
}
else{
    esportes = false;
}
JOptionPane.showMessageDialog(parentComponent: null, "Tecnologia: "+tecnologia+"\nAstronomia: "+astronomia+
    "\nEsportes: "+esportes, title: "Alerta", messageType: JOptionPane.INFORMATION_MESSAGE);
```

Combo Box

- Agora, vamos trabalhar com Combo Box
- Combina uma caixa de texto editável com uma lista suspensa de opções selecionáveis.
- São usados para permitir que os usuários escolham uma opção entre várias em um menu suspenso ou insiram um valor personalizado por meio da caixa de texto.
- Eles são úteis quando você deseja oferecer uma lista de opções predefinidas ou permitir que os usuários insiram dados personalizados.

Combo Box

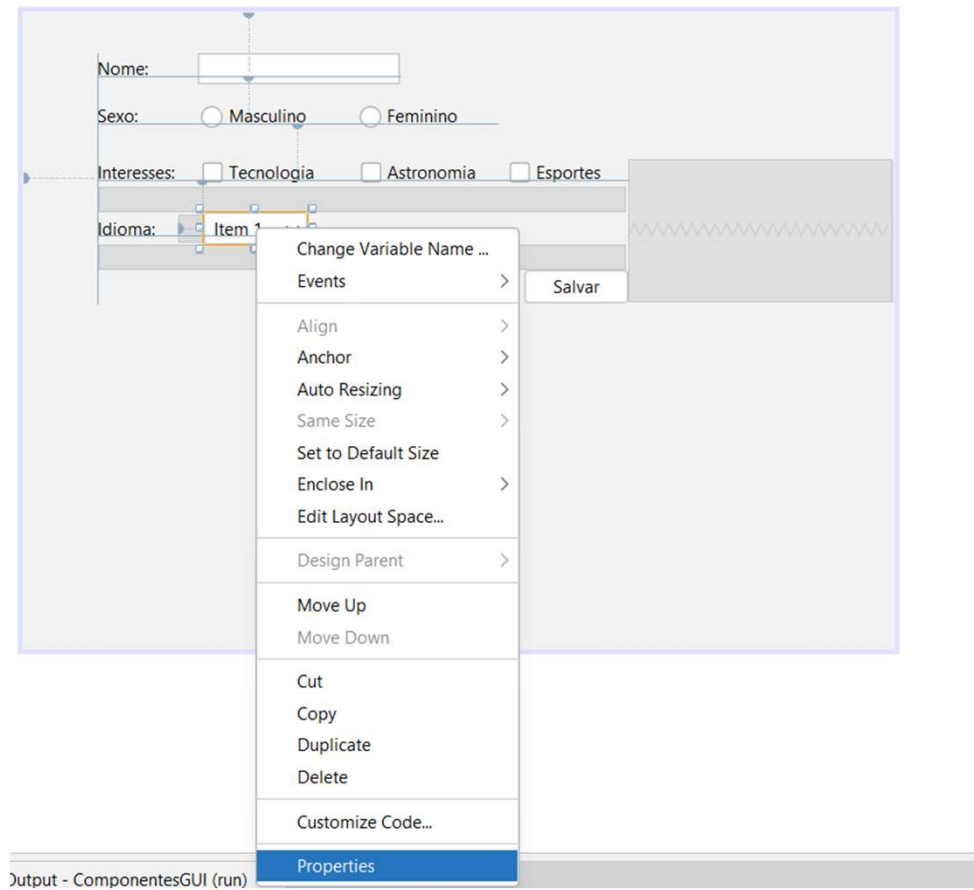
- Adicione um Combo box que vai ser possível selecionar um idioma



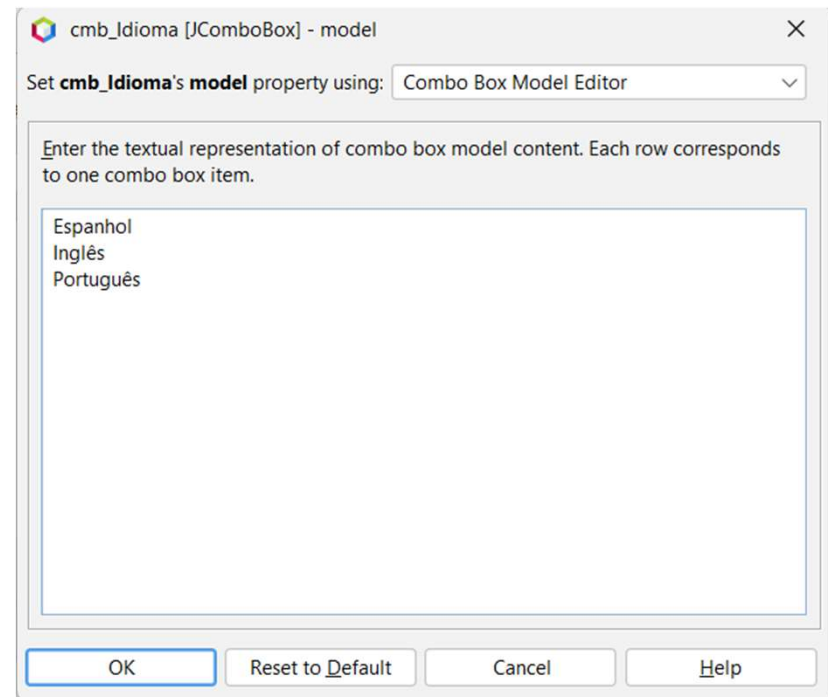
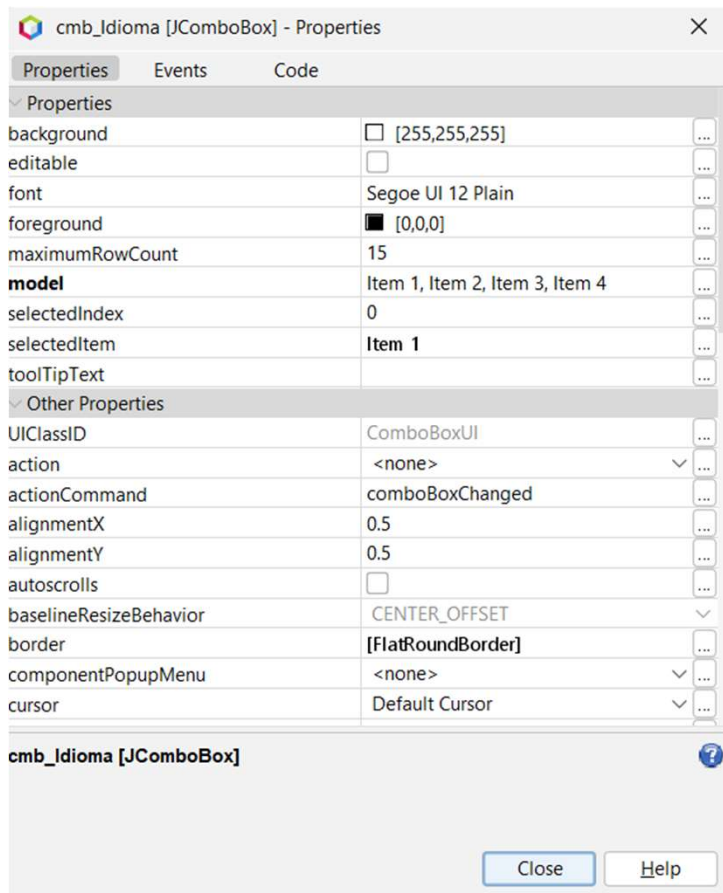
A form with the following fields:

- Nome:
- Sexo: ☐ Masculino ☐ Feminino
- Interesses: ☐ Tecnologia ☐ Astronomia ☐ Esportes
- Idioma:

Combo Box



Combo Box



Combo Box

```
JOptionPane.showMessageDialog(parentComponent: null,  
    "Idioma seleccionado:" +cmb_Idioma.getSelectedItem(),  
    title: "Alerta", messageType: JOptionPane.INFORMATION_MESSAGE);
```

Table

- O JTable em Java é um componente usado para exibir dados em forma de tabela
- Ele permite a criação e exibição de tabelas de dados, semelhantes às que você veria em uma planilha ou em um aplicativo de banco de dados.
- O JTable é altamente personalizável e oferece suporte para uma ampla gama de recursos, como ordenação, seleção de células, edição de células e muito mais.

Table

- Adicione uma tabela:

Nome:

Sexo: ☐ Masculino ☐ Feminino

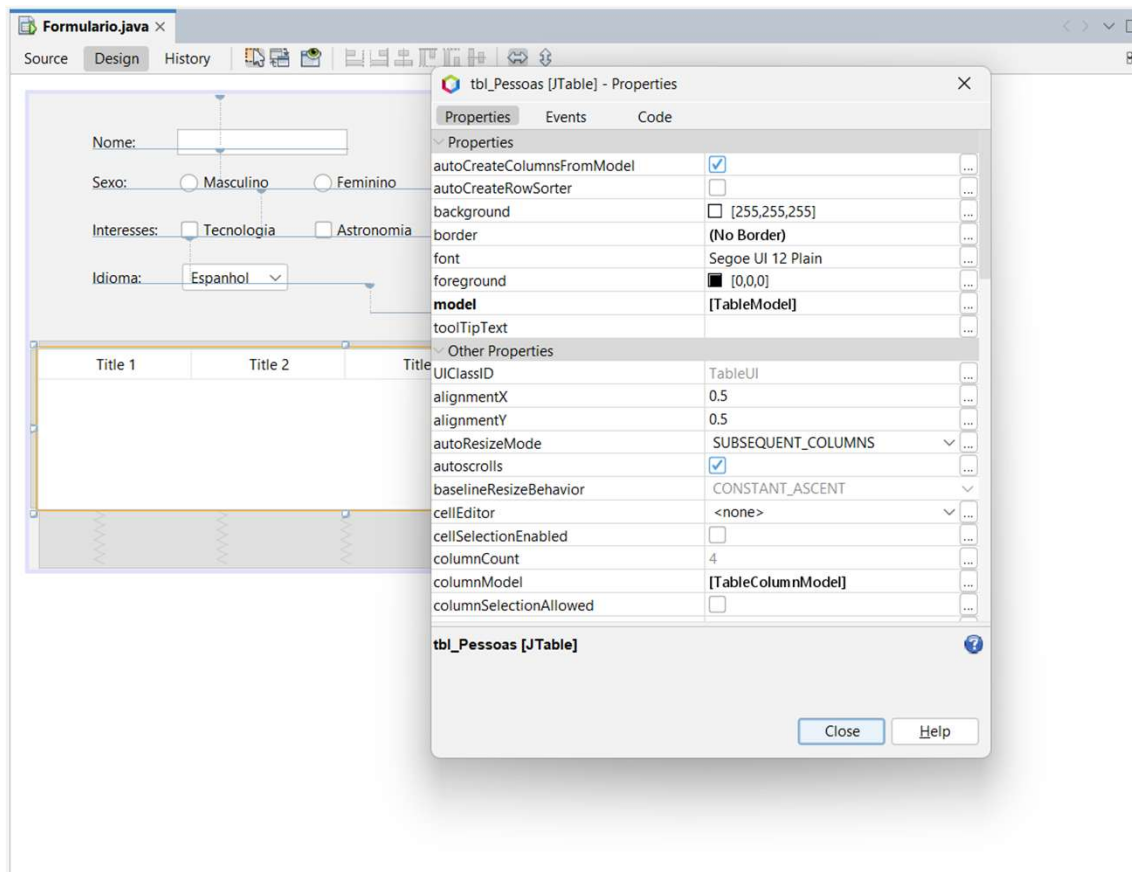
Interesses: ☐ Tecnologia ☐ Astronomia ☐ Esportes

Idioma:

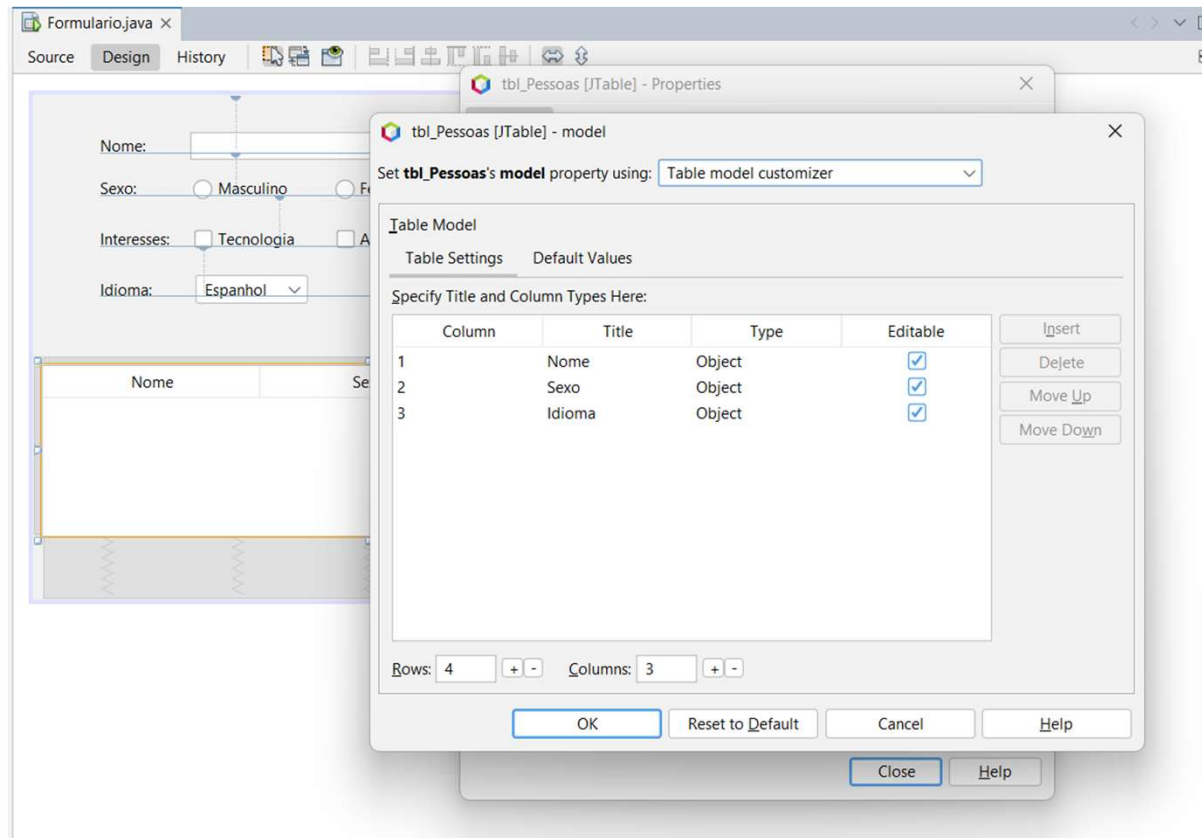
Salvar

Title 1	Title 2	Title 3	Title 4

Table



Table



Table

tbl_Pessoas [JTable] - model

Set **tbl_Pessoas's model** property using: Table model customizer

Table Model

Table Settings Default Values

Specify Title and Column Types Here:

Column	Title	Type	Editable
1	Nome	Object	☒
2	Sexo	Object	☒
3	Idioma	Object	☒

Insert
Delete
Move Up
Move Down

Rows: 0 + - Columns: 3 + -

OK Reset to Default Cancel Help

Table

- Agora, vamos criar uma classe

```
public class Pessoa {  
    public String nome;  
    public char sexo;  
    public String idioma;  
  
    public Pessoa(String nome, char sexo, String idioma){  
        this.nome = nome;  
        this.sexo = sexo;  
        this.idioma = idioma;  
    }  
}
```

Table

- Na ação do botão Salvar, adicione o código:

```
Pessoa p = new Pessoa(nome: txt_Nome.getText(), sexo, (String) cmb_Idioma.getSelectedItem());
```

Table

- Agora vamos adicionar os dados na tabela!
- Para isso, vamos criar um método na classe Pessoa que retornará um objeto com os atributos que irão ser adicionados na tabela
- Adicione o método na classe Pessoa:

```
public Object[] obterDados() {  
    return new Object[] {nome, sexo, idioma};  
}
```

Table

- Na ação do botão Salvar, atualize o código:

```
Pessoa p = new Pessoa(nome: txt_Nome.getText(), sexo, (String) cmb_Idioma.getSelectedItem());  
  
DefaultTableModel tabela = (DefaultTableModel) tbl_Pessoas.getModel();  
tabela.addRow(rowData: p.obterDados());
```

Table

- Ficou bom?
- O que podemos melhorar?
 - Limpe os dados!
 - Como a gente faz?

Table

- Na ação do botão, atualize o código com as linhas abaixo para efetuar a limpeza dos dados.

```
Pessoa p = new Pessoa(nome: txt_Nome.getText(), sexo, (String) cmb_Idioma.getSelectedItem());

DefaultTableModel tabela = (DefaultTableModel) tbl_Pessoas.getModel();
tabela.addRow(rowData: p.obterDados());

txt_Nome.setText(t: "");
btnGrp_Sexo.clearSelection();
chk_Tecnologia.setSelected(b: false);
chk_Astronomia.setSelected(b: false);
chk_Esportes.setSelected(b: false);
cmb_Idioma.setSelectedIndex(anIndex: 0);
```

Atividade

- Criar um cadastro de alunos em um sistema
- Cadastro de Aluno:
 - O usuário deve poder adicionar um novo aluno ao sistema. Cada aluno deve ter os seguintes atributos: Nome completo, Data de nascimento (no formato DD/MM/AAAA), Sexo, Matrícula (um número único), Curso, CPF (Cadastro de Pessoa Física), Endereço completo (rua, número, bairro, cidade, CEP), Estado (combo box), Telefone de contato,
- Utilize classes e adicione em uma lista os dados e apresenta na tabela.

Java e Banco de Dados

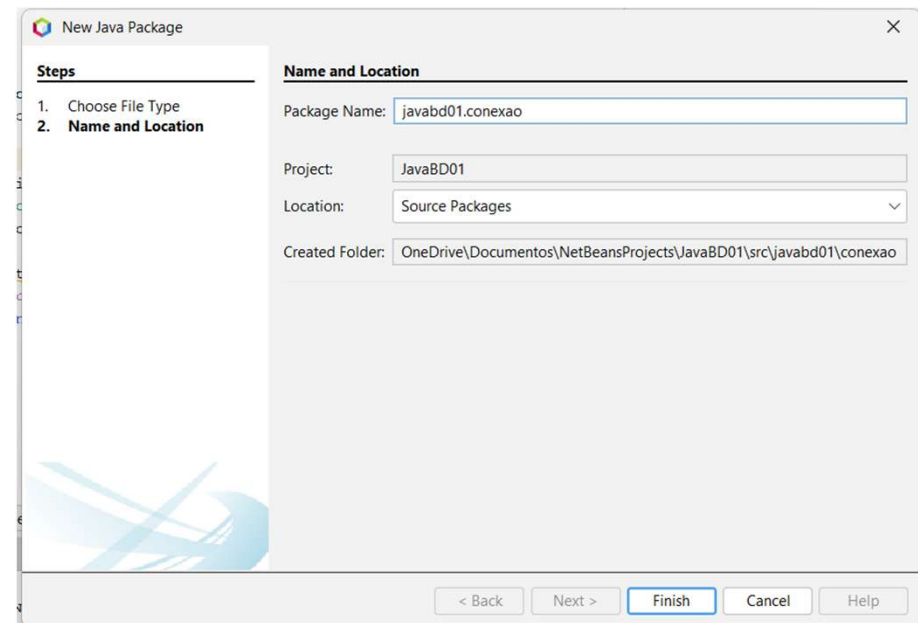


Java Swing

- Agora, vamos trabalhar com Java Swing e Banco de dados
- O banco de dados, utilizaremos o mysql e para utiliza-lo teremos algumas opções..
 - Podemos usar o MySQL nativo, instalado
 - Ou alguma ferramenta para desenvolvedores, como Xampp, Wamp, EasyPHP etc
 - Vocês podem utilizar o que preferir.

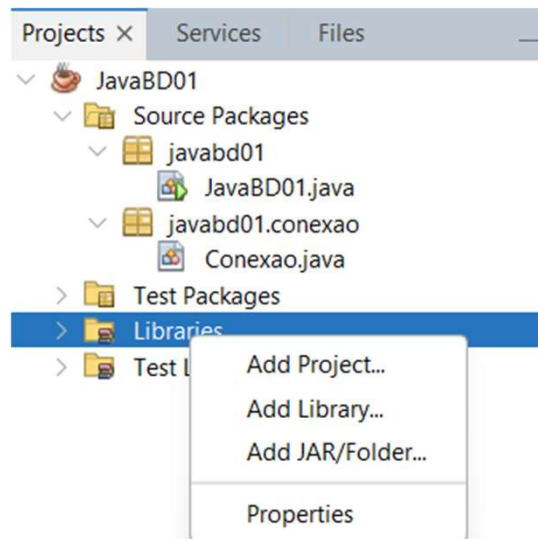
Projeto com banco MySQL

- Crie um projeto usando NetBeans -> Java with Ant -> Java Application.
- Crie um pacote chamado “conexao” e dentro deste pacote, crie uma classe chamada “Conexao”



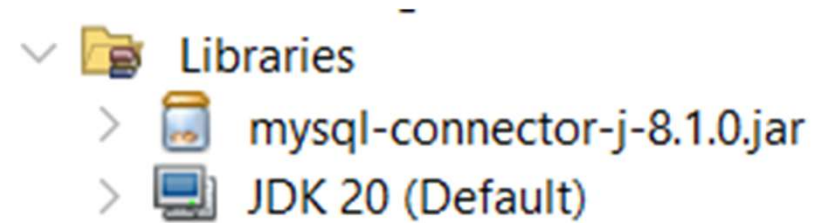
Projeto com banco MySQL

- Agora, clique com o botão direito em “Libraries”
- Selecione Add JAR/Folder



Projeto com banco MySQL

- Agora, clique com o botão direito em “Libraries”
- Selecione Add JAR/Folder
- Selecione o arquivo jar



Projeto com banco MySQL

- Agora na classe Conexao, vamos programar para conectar-se ao banco:

```
public class Conexao {  
    public Connection getConexao() {  
        try{  
  
            Connection conn = DriverManager.getConnection(  
                url:"jdbc:mysql://localhost:3306/bdaula01?useTimezone=true&serverTimezone=UTC",  
                user: "root", password: "1234");  
            System.out.println("Conexão realizada com sucesso!");  
            return conn;  
        }  
        catch (Exception e) {  
            System.out.println("Erro ao conectar no BD"+e.getMessage());  
            return null;  
        }  
    }  
}
```


Projeto com banco MySQL

- E na classe principal, vamos instanciar um objeto da classe e chamar o método getConnection()

```
public class JavaBD01 {  
  
    /**  
     * @param args the command line arguments  
     */  
    public static void main(String[] args) {  
        Conexao c = new Conexao();  
        c.getConnection();  
    }  
}
```

Projeto com banco MySQL

- Se não apareceu mensagem de erro e apareceu somente esta mensagem, conseguimos conectar com o banco:

```
run:
Mon Sep 25 22:12:49 BRT 2023 WARN: Establishing SSL connection without server's identity verification is not recommended. According to MySQL 5.5.45+, 5.6.26+ and 5.7.6+ requirements SSL connection must use X509 Certificates
BUILD SUCCESSFUL (total time: 1 second)
```

Projeto com banco MySQL

- Mas agora, vamos criar dois pacotes novos:
 - Crie um pacote chamado beans
 - Neste pacote, teremos as classes que mapearão as tabelas do banco de dados, ou seja, as classes terão como atributos as colunas da tabela.
 - É interessante em nossos projetos que tenhamos um "bean" para cada tabela do banco.

Projeto com banco MySQL

- Dentro do pacote beans, vamos criar a classe Pessoa:

```
public class Pessoa {  
    private int id;  
    private String nome;  
    private String sexo;  
    private String idioma;
```

```
    public int getId() {  
        return id;  
    }  
    public String getNome() {  
        return nome;  
    }  
    public String getSexo() {  
        return sexo;  
    }  
    public String getIdioma() {  
        return idioma;  
    }  
    public void setId(int id) {  
        this.id = id;  
    }  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
    public void setSexo(String sexo) {  
        this.sexo = sexo;  
    }  
    public void setIdioma(String idioma) {  
        this.idioma = idioma;  
    }  
}
```

Projeto com banco MySQL

- Mas agora, vamos criar dois pacotes novos:
 - E outro pacote chamado dao:
 - DAO = Data Access Object ou Objeto de Acesso aos Dados
 - Neste pacote, teremos as classes responsáveis por persistir os dados (inserção, edição, exclusão e pesquisa) da aplicação nas tabelas.
 - Resumindo, é ela que executará os códigos SQL no banco.
 - É importante que tenhamos também um DAO para cada tabela.

Projeto com banco MySQL

- E no pacote DAO, vamos fazer o insert no banco:

```
public class PessoaDAO {
    private Conexao conexao;
    private Connection conn;

    public PessoaDAO() {
        this.conexao = new Conexao();
        this.conn = this.conexao.getConexao();
    }

    public void inserir (Pessoa pessoa){
        String sql = "INSERT INTO pessoa (nome, sexo, idioma) VALUES (?, ?, ?)";

        try {
            PreparedStatement stmt = this.conn.prepareStatement(string:sql);
            stmt.setString(i: 1, string:pessoa.getNome());
            stmt.setString(i: 2, string:pessoa.getSexo());
            stmt.setString(i: 3, string:pessoa.getIdioma());

            stmt.execute();
        } catch (SQLException ex) {
            System.out.println("Erro ao inserir pessoa: "+ex.getMessage());
        }
    }
}
```

Projeto com banco MySQL

```
import beans.Pessoa;
import javabd01.conexao.Conexao;
import java.sql.Connection;
import java.sql.*;
import java.sql.SQLException;
import java.util.logging.Level;
import java.util.logging.Logger;
```

```
public class PessoaDAO {
    private Conexao conexao;
    private Connection conn;

    public PessoaDAO() {
        this.conexao = new Conexao();
        this.conn = this.conexao.getConexao();
    }

    public void inserir (Pessoa pessoa) {
        String sql = "INSERT INTO pessoa (nome, sexo, idioma) VALUES (?, ?, ?)";

        try {
            PreparedStatement stmt = this.conn.prepareStatement(string:sql);
            stmt.setString(i: 1, string:pessoa.getNome());
            stmt.setString(i: 2, string:pessoa.getSexo());
            stmt.setString(i: 3, string:pessoa.getIdioma());

            stmt.execute();
        } catch (SQLException ex) {
            System.out.println("Erro ao inserir pessoa: "+ex.getMessage());
        }
    }
}
```

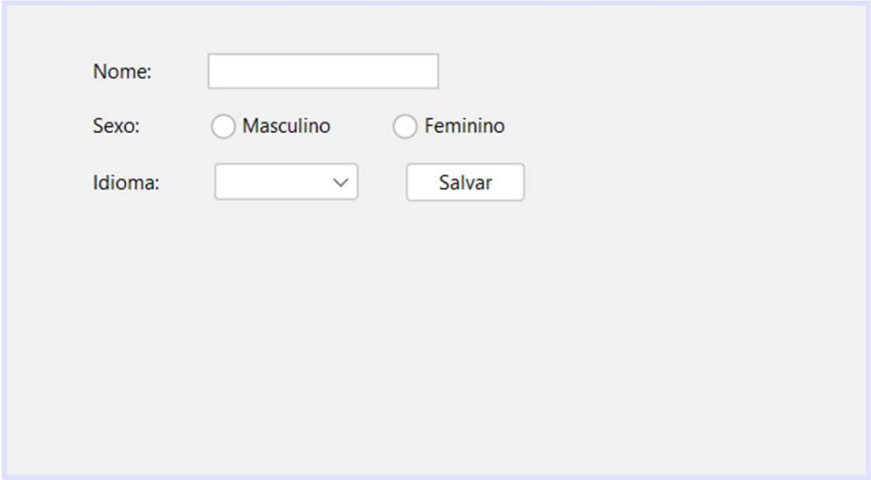
Projeto com banco MySQL

- E na main, vamos fazer somente um teste:

```
public static void main(String[] args) {  
  
    Pessoa p = new Pessoa();  
    p.setNome(nome: "Ricardo");  
    p.setSexo(sexo: "M");  
    p.setIdioma(idioma: "PORTUGUÊS");  
  
    PessoaDAO pDAO = new PessoaDAO();  
    pDAO.inserir(pessoa:p);  
}
```


Projeto com banco MySQL

- Vamos fazer a interface?
- Faça a interface e faça com que seja cadastrado pessoas a partir dela no banco de dados



A user registration form with a light gray background and a thin blue border. It contains three input fields: a text box for 'Nome:', a radio button group for 'Sexo:' with options 'Masculino' and 'Feminino', and a dropdown menu for 'Idioma:'. A 'Salvar' button is positioned to the right of the dropdown menu.

Nome:

Sexo: ☐ Masculino ☐ Feminino

Idioma:

Projeto com banco MySQL

- Vamos fazer a interface?
- Atividade 1: Faça a interface e faça com que seja cadastrado pessoas a partir dela no banco de dados

```
private void btn_SalvarActionPerformed(java.awt.event.ActionEvent evt) {  
    if(rdo_Masculino.isSelected()){  
        sexo = "M";  
        /* JOptionPane.showMessageDialog(null,  
            "Sexo selecionado: "+sexo, "Alerta!" , JOptionPane.OK_OPTION);*/  
    }  
    else if(rdo_Feminino.isSelected()){  
        sexo = "F";  
        /*JOptionPane.showMessageDialog(null,  
            "Sexo selecionado: "+sexo, "Alerta!" , JOptionPane.OK_OPTION);*/  
    }  
    Pessoa p = new Pessoa();  
    p.setNome(nome: txt_Nome.getText());  
    p.setSexo(sexo);  
    p.setIdioma(idioma: cmb_Idioma.getSelectedItem().toString());  
  
    PessoaDAO pDAO = new PessoaDAO();  
    pDAO.inserir(pessoa:p);  
    limparFormulario();  
}
```

Projeto com banco MySQL

```
private void btn_SalvarActionPerformed(java.awt.event.ActionEvent evt) {  
    if(rdo_Masculino.isSelected()){  
        sexo = "M";  
        /* JOptionPane.showMessageDialog(null,  
            "Sexo selecionado: "+sexo, "Alerta!" , JOptionPane.OK_OPTION);*/  
    }  
    else if(rdo_Feminino.isSelected()){  
        sexo = "F";  
        /*JOptionPane.showMessageDialog(null,  
            "Sexo selecionado: "+sexo, "Alerta!" , JOptionPane.OK_OPTION);*/  
    }  
    Pessoa p = new Pessoa();  
    p.setNome(nome: txt_Nome.getText());  
    p.setSexo(sexo);  
    p.setIdioma(idioma: cmb_Idioma.getSelectedItem().toString());  
  
    PessoaDAO pDAO = new PessoaDAO();  
    pDAO.inserir(pessoa:p);  
    limparFormulario();  
}
```

Projeto com banco MySQL

```
private void limparFormulario() {  
    txt_Nome.setText(t: "");  
    btnGrp_Sexo.clearSelection();  
    cmb_Idioma.setSelectedIndex(anIndex: 0);  
}
```

Projeto com banco MySQL

- Vamos criar uma consulta de pessoa pelo ID.
- Para isso, precisaremos trabalhar na classe PessoaDAO inserindo o método getPessoa

```
public Pessoa getPessoa(int id){
    String sql = "SELECT * FROM pessoa WHERE id = ?";
    try {
        PreparedStatement stmt = conn.prepareStatement(string:sql,1: ResultSet.TYPE_SCROLL_INSENSITIVE,
            11: ResultSet.CONCUR_UPDATABLE);
        //1° parâmetro é o SQL
        //2° parâmetro é o tipo do ResultSet -
        //ResultSet scroll, ou seja, o cursor se move para frente ou para trás.
        //Este tipo de ResultSet é sensível às alterações feitas no banco de dados, ou seja, as
        //modificações feitas no banco de dados são refletidas no ResultSet.
        //3° parâmetro é sobre os parâmetros de concorrência - pode ser "read only" ou atualizável
        stmt.setInt(1:1, 11: id);
        ResultSet rs = stmt.executeQuery(); //obtenho o retorno da consulta e armazeno no ResultSet
        Pessoa p = new Pessoa(); //Preparo um objeto que vou armazenar a consulta
        //Primeiramente, vamos posicionar o retorno da consulta (ResultSet) na primeira posição da consulta
        //Em alguns casos, a consulta terá mais de um resultado de retorno
        rs.first();
        p.setId(id);
        p.setNome(nome: rs.getString(string:"nome"));
        p.setSexo(sexo: rs.getString(string:"sexo"));
        p.setIdioma(idioma:rs.getString(string:"idioma"));
        return p;
    } catch (SQLException ex) {
        System.out.println("Erro ao consultar pessoa: "+ex.getMessage());
        return null;
    }
}
```

```

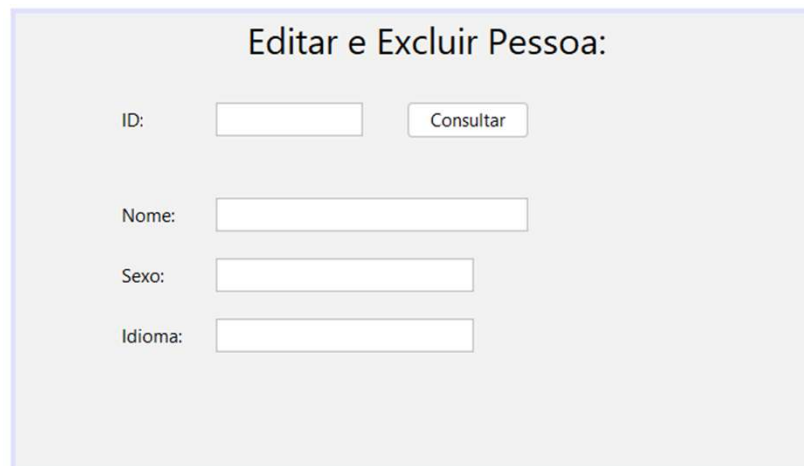
public Pessoa getPessoa(int id){
    String sql = "SELECT * FROM pessoa WHERE id = ?";
    try {
        PreparedStatement stmt = conn.prepareStatement(string:sql,i: ResultSet.TYPE_SCROLL_INSENSITIVE,
            il: ResultSet.CONCUR_UPDATABLE);
        //1º parâmetro é o SQL
        //2º parâmetro é o tipo do ResultSet -
        //ResultSet scroll, ou seja, o cursor se move para frente ou para trás.
        //Este tipo de ResultSet é sensível às alterações feitas no banco de dados, ou seja, as
        //modificações feitas no banco de dados são refletidas no ResultSet.
        //3º parâmetro é sobre os parâmetros de concorrência - pode ser "read only" ou atualizável
        stmt.setInt(i: 1, il: id);
        ResultSet rs = stmt.executeQuery(); //obtenho o retorno da consulta e armazeno no ResultSet
        Pessoa p = new Pessoa(); //Preparo um objeto que vou armazenar a consulta
        //Primeiramente, vamos posicionar o retorno da consulta (ResultSet) na primeira posição da consulta
        //Em alguns casos, a consulta terá mais de um resultado de retorno
        rs.first();
        p.setId(id);
        p.setNome(nome: rs.getString(string:"nome"));
        p.setSexo(sexo: rs.getString(string:"sexo"));
        p.setIdioma(idioma: rs.getString(string:"idioma"));
        return p;

    } catch (SQLException ex) {
        System.out.println("Erro ao consultar pessoa: "+ex.getMessage());
        return null;
    }
}

```

Projeto com banco MySQL

- E para consultar, precisaremos ter uma interface.
- Ela servirá para consultar (e daqui a pouco, editar e excluir o registro)
- Nesta interface, inicialmente, teremos estes campos:



Editar e Excluir Pessoa:

ID:

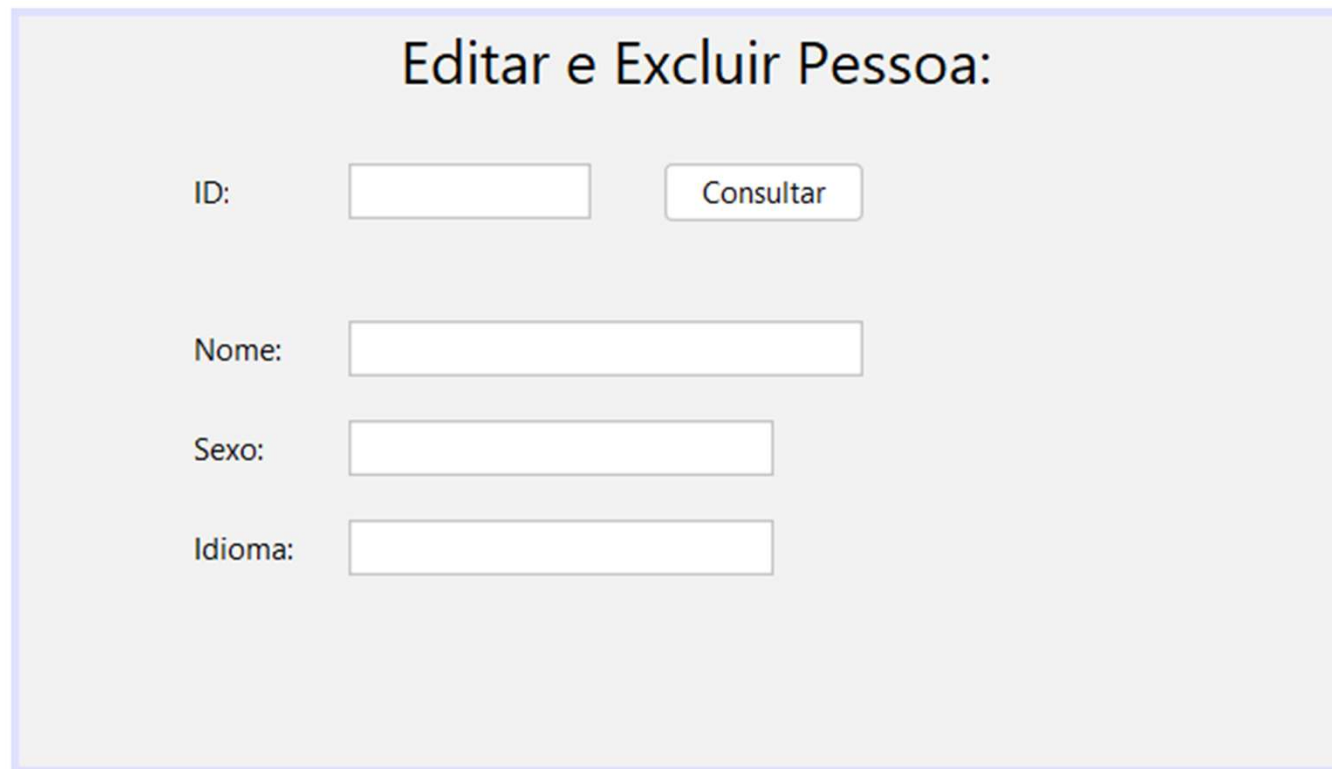
Nome:

Sexo:

Idioma:

Projeto com banco MySQL

- Agora, vamos criar um outro formulário (Jframe Form)



Editar e Excluir Pessoa:

ID:

Nome:

Sexo:

Idioma:

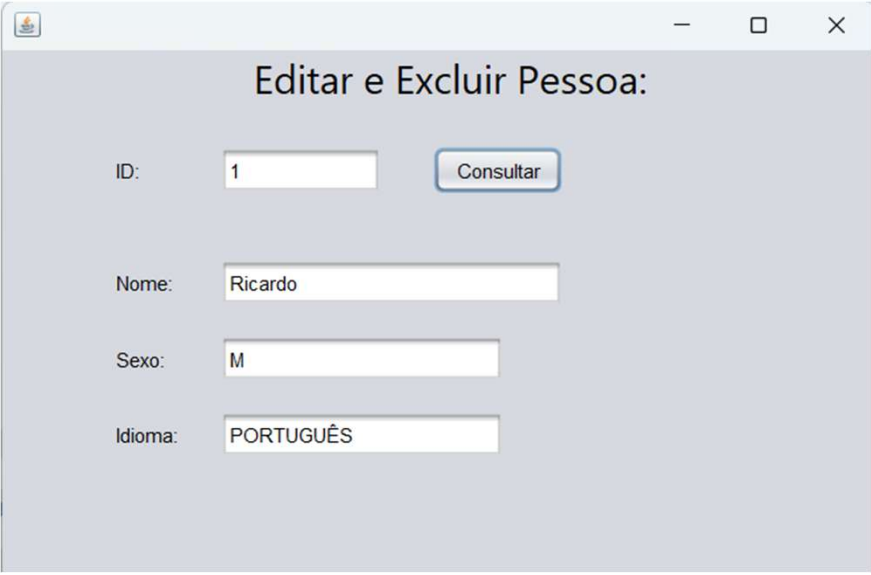
Projeto com banco MySQL

- E no clique do botão consultar, vamos chamar o método getPessoa da classe PessoaDAO:

```
private void btn_ConsultarActionPerformed(java.awt.event.ActionEvent evt) {  
    int idPessoa = Integer.parseInt(s: txt_IDPessoa.getText());  
    PessoaDAO pDAO = new PessoaDAO();  
  
    Pessoa p = pDAO.getPessoa(id: idPessoa);  
    if(p == null){  
        txt_Nome.setText(t: "");  
        txt_Sexo.setText(t: "");  
        txt_Idioma.setText(t: "");  
        JOptionPane.showMessageDialog(parentComponent: this, message: "Pessoa não encontrada!");  
    }  
    else{  
        txt_Nome.setText(t: p.getNome());  
        txt_Sexo.setText(t: p.getSexo());  
        txt_Idioma.setText(t: p.getIdioma());  
    }  
}
```

Projeto com banco MySQL

- E deve funcionar desta forma:



The screenshot shows a web application window with the title "Editar e Excluir Pessoa:". The window contains a form with the following fields and controls:

- ID:** A text input field containing the value "1". To its right is a button labeled "Consultar".
- Nome:** A text input field containing the value "Ricardo".
- Sexo:** A text input field containing the value "M".
- Idioma:** A text input field containing the value "PORTUGUÊS".

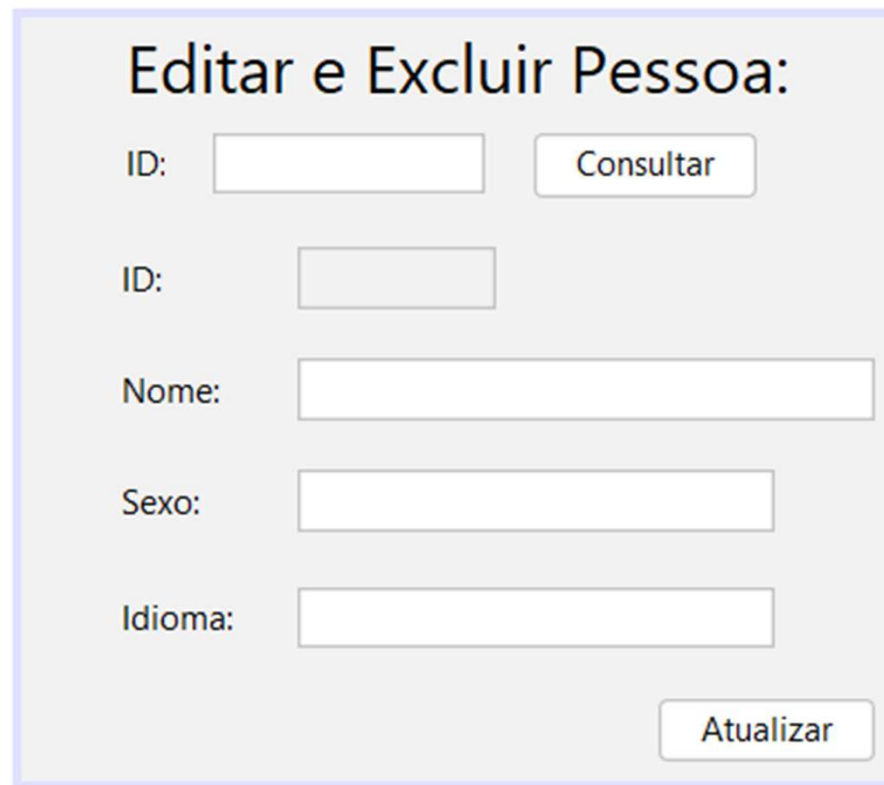
Projeto com banco MySQL

- E agora, podemos implementar o editar (update) e para isso, vamos trabalhar novamente na classe PessoaDAO:

```
public void editar(Pessoa pessoa) {  
    try {  
        String sql = "UPDATE pessoa set nome=?, sexo=?, idioma=? WHERE id=?";  
  
        PreparedStatement stmt = conn.prepareStatement(string:sql);  
        stmt.setString(i: 1, string:pessoa.getNome());  
        stmt.setString(i: 2, string:pessoa.getSexo());  
        stmt.setString(i: 3, string:pessoa.getIdioma());  
        stmt.setInt(i: 4, i1: pessoa.getId());  
        stmt.execute();  
    } catch (SQLException ex) {  
        System.out.println("Erro ao atualizar pessoa: "+ex.getMessage());  
    }  
}
```

Projeto com banco MySQL

- E vamos atualizar a nossa interface gráfica com os seguintes componentes:



Editar e Excluir Pessoa:

ID:

ID:

Nome:

Sexo:

Idioma:

Projeto com banco MySQL

- E no botão atualizar, vamos colocar o seguinte código:

```
private void btn_AtualizarActionPerformed(java.awt.event.ActionEvent evt) {  
    Pessoa p = new Pessoa();  
    p.setId(id: Integer.parseInt(s: txt_ID.getText()));  
    p.setNome(nome: txt_Nome.getText());  
    p.setSexo(sexo: txt_Sexo.getText());  
    p.setIdioma(idioma: txt_Idioma.getText());  
  
    PessoaDAO pDAO = new PessoaDAO();  
    pDAO.editar(pessoa:p);  
    limparFormulario();  
}
```

```
private void limparFormulario() {  
    txt_ID.setText(t: "");  
    txt_Nome.setText(t: "");  
    txt_Sexo.setText(t: "");  
    txt_Idioma.setText(t: "");  
}
```

Projeto com banco MySQL

- E agora, vamos criar um método de exclusão dentro da classe PessoaDAO:

```
public void excluir(int id){  
    try {  
        String sql = "delete from pessoa WHERE id=?";  
  
        PreparedStatement stmt = conn.prepareStatement(string:sql);  
        stmt.setInt(1, id);  
        stmt.execute();  
    } catch (SQLException ex) {  
        System.out.println("Erro ao excluir pessoa: "+ex.getMessage());  
    }  
}
```

Projeto com banco MySQL

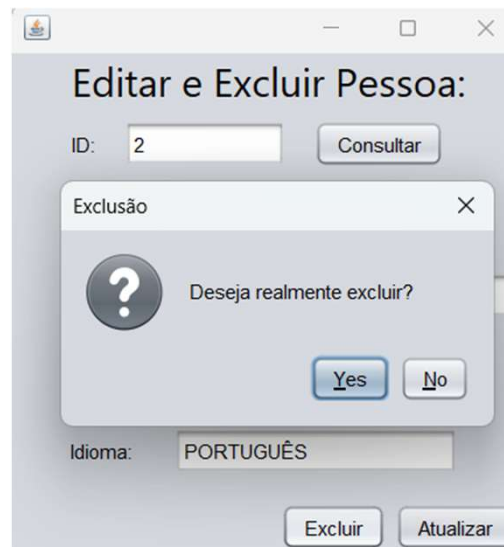
- Após, vamos adicionar um botão na nossa interface que irá chamar a exclusão:

```
private void btn ExcluirActionPerformed(java.awt.event.ActionEvent evt) {  
    int resposta = JOptionPane.showConfirmDialog(parentComponent: null, message: "Deseja realmente excluir?",  
        title: "Exclusão", optionType: JOptionPane.YES_NO_OPTION);  
    if (resposta == JOptionPane.YES_OPTION) {  
        PessoaDAO pDAO = new PessoaDAO();  
        pDAO.excluir(id: Integer.parseInt(s: txt_ID.getText()));  
        limparFormulario();  
    }  
}
```

```
}
```

Projeto com banco MySQL

- E o resultado deve ser este:



Projeto com banco MySQL

- E agora, vamos melhorar o nosso editar e excluir, exclua os TextField dos campos sexo e idioma e adiciona radio button para o sexo e combo box para o idioma:

Editar e Excluir Pessoa:

ID:

ID:

Nome:

Sexo: ☐ Masculino ☐ Feminino

Idioma:

Projeto com banco MySQL

- Depois, vamos alterar no consultar:

```
private void btn_ConsultarActionPerformed(java.awt.event.ActionEvent evt) {  
    int idPessoa = Integer.parseInt(s: txt_IDPessoa.getText());  
    PessoaDAO pDAO = new PessoaDAO();  
  
    Pessoa p = pDAO.getPessoa(id: idPessoa);  
    if(p == null){  
        limparFormulario();  
        JOptionPane.showMessageDialog(parentComponent: this, message: "Pessoa não encontrada!");  
    }  
    else{  
        txt_ID.setText(t: String.valueOf(i: p.getId()));  
        txt_Nome.setText(t: p.getNome());  
        //txt_Sexo.setText(p.getSexo());  
        if(p.getSexo().equals(anObject: "M")){  
            rdo_Masculino.setSelected(b: true);  
        }  
        else{  
            rdo_Feminino.setSelected(b: true);  
        }  
        //txt_Idioma.setText(p.getIdioma());  
        cmb_Idioma.setSelectedItem(anObject: p.getIdioma());  
    }  
}
```

```
private void btn_ConsultarActionPerformed(java.awt.event.ActionEvent evt) {  
    int idPessoa = Integer.parseInt(s: txt_IDPessoa.getText());  
    PessoaDAO pDAO = new PessoaDAO();  
  
    Pessoa p = pDAO.getPessoa(id: idPessoa);  
    if(p == null){  
        limparFormulario();  
        JOptionPane.showMessageDialog(parentComponent: this, message: "Pessoa não encontrada!");  
    }  
    else{  
        txt_ID.setText(t: String.valueOf(i: p.getId()));  
        txt_Nome.setText(t: p.getNome());  
        //txt_Sexo.setText(p.getSexo());  
        if(p.getSexo().equals(anObject: "M")){  
            rdo_Masculino.setSelected(b: true);  
        }  
        else{  
            rdo_Feminino.setSelected(b: true);  
        }  
        //txt_Idioma.setText(p.getIdioma());  
        cmb_Idioma.setSelectedItem(anObject: p.getIdioma());  
    }  
}
```

Projeto com banco MySQL

- Depois, vamos alterar no atualizar:

```
private void btn_AtualizarActionPerformed(java.awt.event.ActionEvent evt) {  
    String sexo = null;  
    if(rdo_Masculino.isSelected()){  
        sexo = "M";  
    }  
    else if(rdo_Feminino.isSelected()){  
        sexo = "F";  
    }  
    Pessoa p = new Pessoa();  
    p.setId(id: Integer.parseInt(s: txt_ID.getText()));  
    p.setNome(nome: txt_Nome.getText());  
    p.setSexo(sexo);  
    p.setIdioma(idioma: cmb_Idioma.getSelectedItem().toString());  
  
    PessoaDAO pDAO = new PessoaDAO();  
    pDAO.editar(pessoa:p);  
    limparFormulario();  
}
```

```
private void btn_AtualizarActionPerformed(java.awt.event.ActionEvent evt) {  
    String sexo = null;  
    if(rdo_Masculino.isSelected()){  
        sexo = "M";  
    }  
    else if(rdo_Feminino.isSelected()){  
        sexo = "F";  
    }  
    Pessoa p = new Pessoa();  
    p.setId(id: Integer.parseInt(s: txt_ID.getText()));  
    p.setNome(nome: txt_Nome.getText());  
    p.setSexo(sexo);  
    p.setIdioma(idioma: cmb_Idioma.getSelectedItem().toString());  
  
    PessoaDAO pDAO = new PessoaDAO();  
    pDAO.editar(pessoa:p);  
    limparFormulario();  
}
```

Projeto com banco MySQL

- Depois, consequentemente alterar o método limparFormulario:

```
private void limparFormulario() {  
    txt_IDPessoa.setText(t: "");  
    txt_ID.setText(t: "");  
    txt_Nome.setText(t: "");  
    btnGrp_Sexo.clearSelection();  
    cmb_Idioma.setSelectedIndex(anIndex: 0);  
}
```

Projeto com banco MySQL

- E agora, vamos fazer a consulta de todos as pessoas cadastradas no banco?
- Para isso, vamos lidar novamente na classe PessoaDAO:

```
public List<Pessoa> getPessoas(){

    String sql = "SELECT * FROM pessoa";
    try {

        PreparedStatement stmt = conn.prepareStatement(string:sql,i: ResultSet.TYPE_SCROLL_INSENSITIVE,
            il: ResultSet.CONCUR_UPDATABLE);

        ResultSet rs = stmt.executeQuery(); //obtenho o retorno da consulta e armazeno no ResultSet
        List<Pessoa> listaPessoas = new ArrayList(); //Preparo uma lista de objetos que vou armazenar a consulta
        //Percorre rs e salvar as informações dentro de um objeto Pessoa e depois adiciona na lista
        while (rs.next()){
            Pessoa p = new Pessoa();
            p.setId(id: rs.getInt(string:"id"));
            p.setNome(nome: rs.getString(string:"nome"));
            p.setSexo(sexo: rs.getString(string:"sexo"));
            p.setIdioma(idioma:rs.getString(string:"idioma"));
            listaPessoas.add(e: p);
        }
        return listaPessoas;
    } catch (SQLException ex) {
        System.out.println("Erro ao consultar todas as pessoas: "+ex.getMessage());
        return null;
    }
}
```

Projeto com banco MySQL

```
public List<Pessoa> getPessoas() {

    String sql = "SELECT * FROM pessoa";
    try {

        PreparedStatement stmt = conn.prepareStatement(string:sql,i: ResultSet.TYPE_SCROLL_INSENSITIVE,
            il: ResultSet.CONCUR_UPDATABLE);

        ResultSet rs = stmt.executeQuery(); //obtenho o retorno da consulta e armazeno no ResultSet
        List<Pessoa> listaPessoas = new ArrayList(); //Preparo uma lista de objetos que vou armazenar a consulta
        //Percorre rs e salvar as informações dentro de um objeto Pessoa e depois adiciona na lista
        while (rs.next()){
            Pessoa p = new Pessoa();
            p.setId(id: rs.getInt(string:"id"));
            p.setNome(nome: rs.getString(string:"nome"));
            p.setSexo(sexo: rs.getString(string:"sexo"));
            p.setIdioma(idioma:rs.getString(string:"idioma"));
            listaPessoas.add(e: p);
        }
        return listaPessoas;
    } catch (SQLException ex) {
        System.out.println("Erro ao consultar todas as pessoas: "+ex.getMessage());
        return null;
    }
}
```


Projeto com banco MySQL

- Depois, precisaremos ter uma tela (Jframe Form) desta forma (chamada RelatorioPessoas):

Relatório das Pessoas			
ID	Nome	Sexo	Idioma

Projeto com banco MySQL

- Depois, precisaremos de um método para preencher a tabela com uma lista de Pessoas que é retornada do método que temos no DAO:

```
public void preencheTabela() {  
    PessoaDAO pDAO = new PessoaDAO();  
    List<Pessoa> listaPessoas = pDAO.getPessoas();  
  
    DefaultTableModel tabelaPessoas = (DefaultTableModel) tbl_Pessoas.getModel();  
  
    //Agora, precisamos percorrer a lista de pessoas que foi consultada e adicionar na tabela:  
    for(Pessoa p: listaPessoas){  
        Object[] obj = new Object[]{  
            p.getId(),  
            p.getNome(),  
            p.getSexo(),  
            p.getIdioma()};  
        tabelaPessoas.addRow(rowData: obj);  
    }  
}
```

Projeto com banco MySQL

- Depois, no construtor do JFrame Form criado, precisamos chamar este método:

```
public RelatorioPessoas() {  
    initComponents();  
    preencheTabela();  
}
```

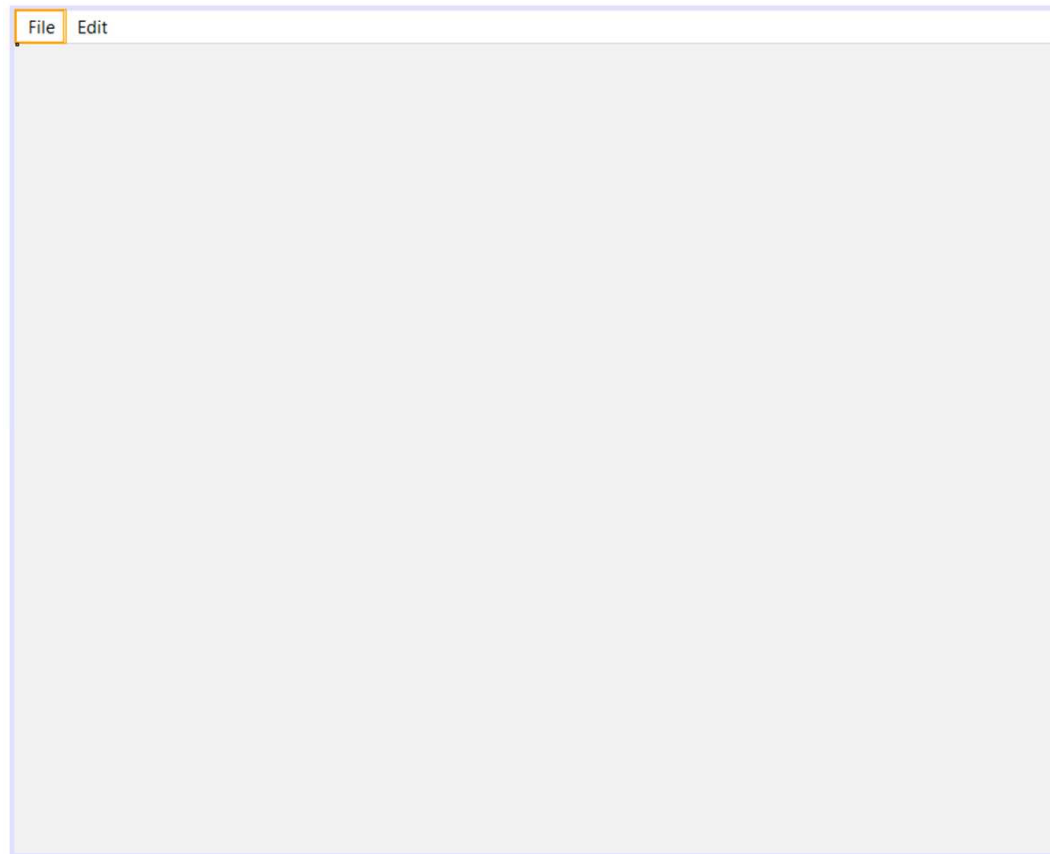


ID	Nome	Sexo	Idioma
2	Ricardo Frohlich	M	PORTUGUÊS
5	Teste	F	Inglês
6	Rafael	M	Espanhol
7	Sabrina	F	Português
8	Ana Paula	F	Inglês

Projeto com banco MySQL

- Agora, vamos unir todas as telas a partir de um menu principal?
 - Crie um formulário novo (Jframe Form) chamado MenuPrincipal
 - Adicione um componente chamado MenuBar

Projeto com banco MySQL



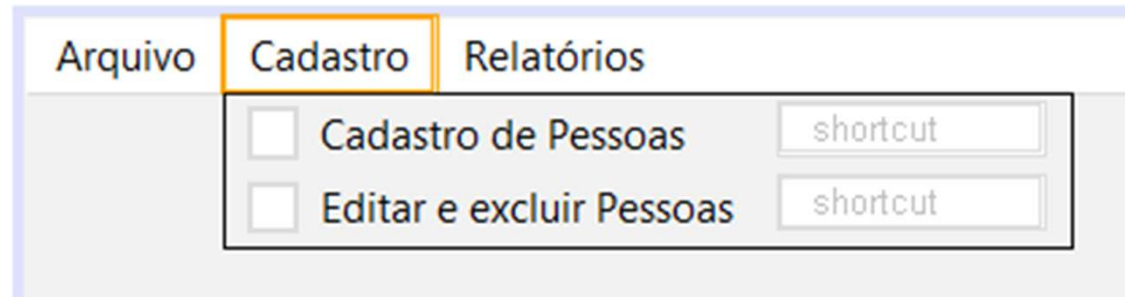
Projeto com banco MySQL

- Aí vocês podem renomear e/ adicionar outros itens
 - Para adicionar novo itens, basta adicionar outros itens de Menu
 - Adicione um Menu chamado Cadastro
 - Adicione um Menu chamado Relatórios



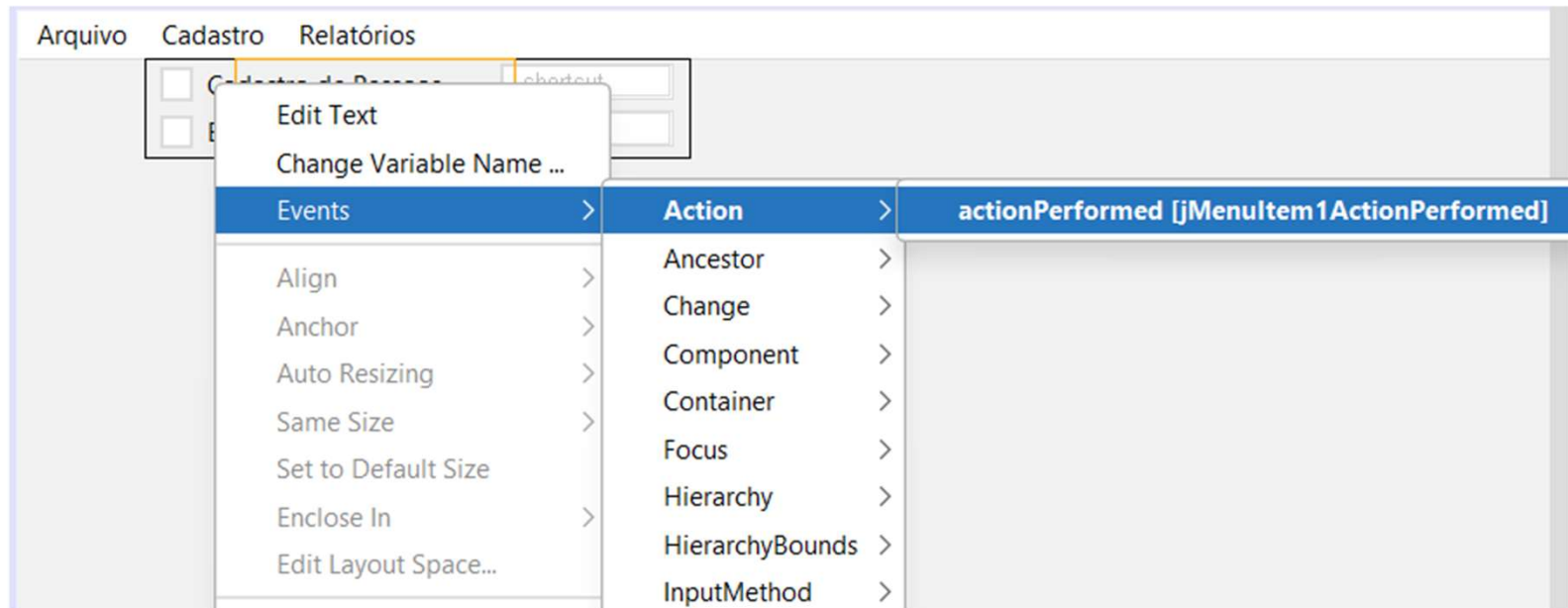
Projeto com banco MySQL

- Após, dentro do Cadastro, adicione dois “Menu Item” do cadastro chamados:



Projeto com banco MySQL

- Após, vamos chamar a tela de cadastro de pessoas:



Projeto com banco MySQL

- Após, vamos chamar a tela de cadastro de pessoas:

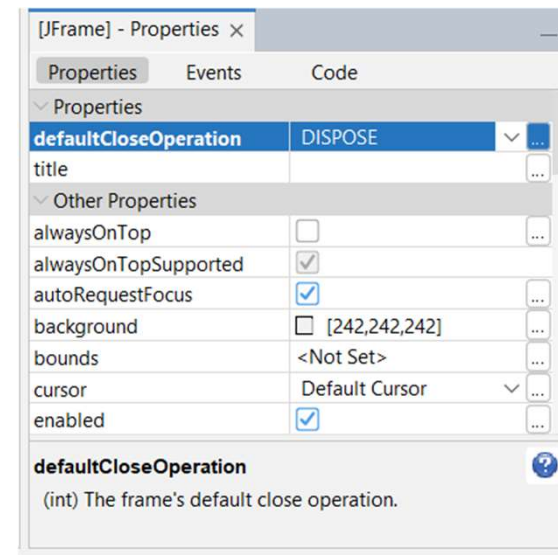
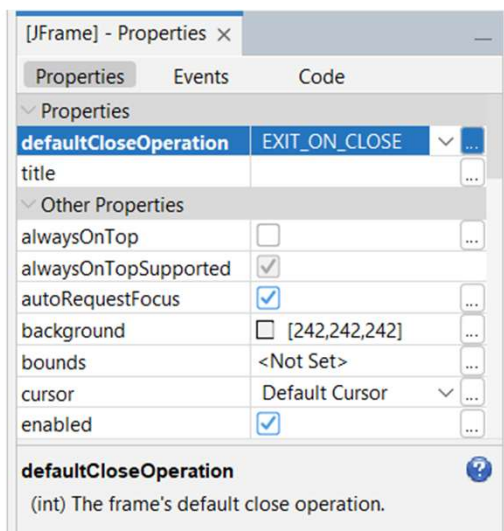
```
private void menu_CadPessoaActionPerformed(java.awt.event.ActionEvent evt) {  
    CadastraPessoa cadPessoa = new CadastraPessoa();  
    cadPessoa.setVisible(b: true);  
}
```

Projeto com banco MySQL

- Testaram? Funcionou?
 - Quando fecha o cadastro, o que acontece?

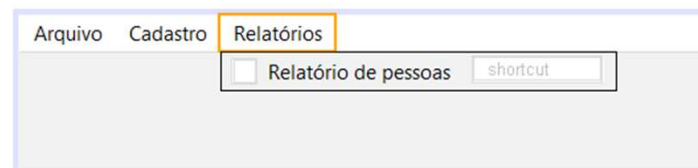
Projeto com banco MySQL

- Vamos resolver isso:
- Vai na tela do Cadastro de pessoas e vá em propriedades e procure pela propriedade DefaultCloseOperation e mude para DISPOSE



Projeto com banco MySQL

- Agora, façam com o restante:

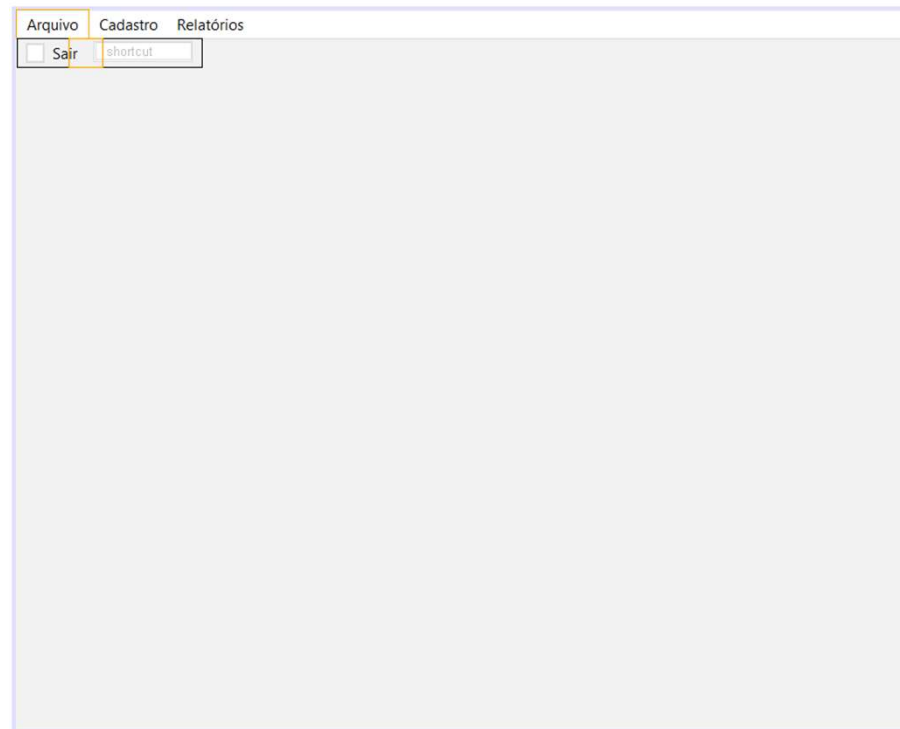


```
private void menu EditarActionPerformed(java.awt.event.ActionEvent evt) {  
    ConsultaPessoa consPessoa = new ConsultaPessoa();  
    consPessoa.setVisible(b: true);  
}
```

```
private void menu_RelatorioActionPerformed(java.awt.event.ActionEvent evt) {  
    RelatorioPessoas relPessoa = new RelatorioPessoas();  
    relPessoa.setVisible(b: true);  
}
```

Projeto com banco MySQL

- Agora, no arquivo, adicione o botão item de menu Sair:



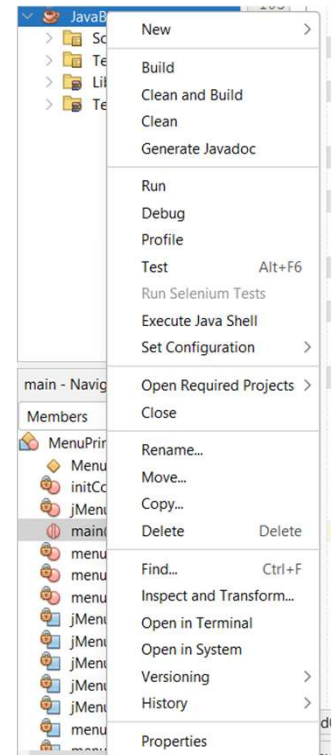
Projeto com banco MySQL

- Agora, no arquivo, adicione o botão item de menu Sair:

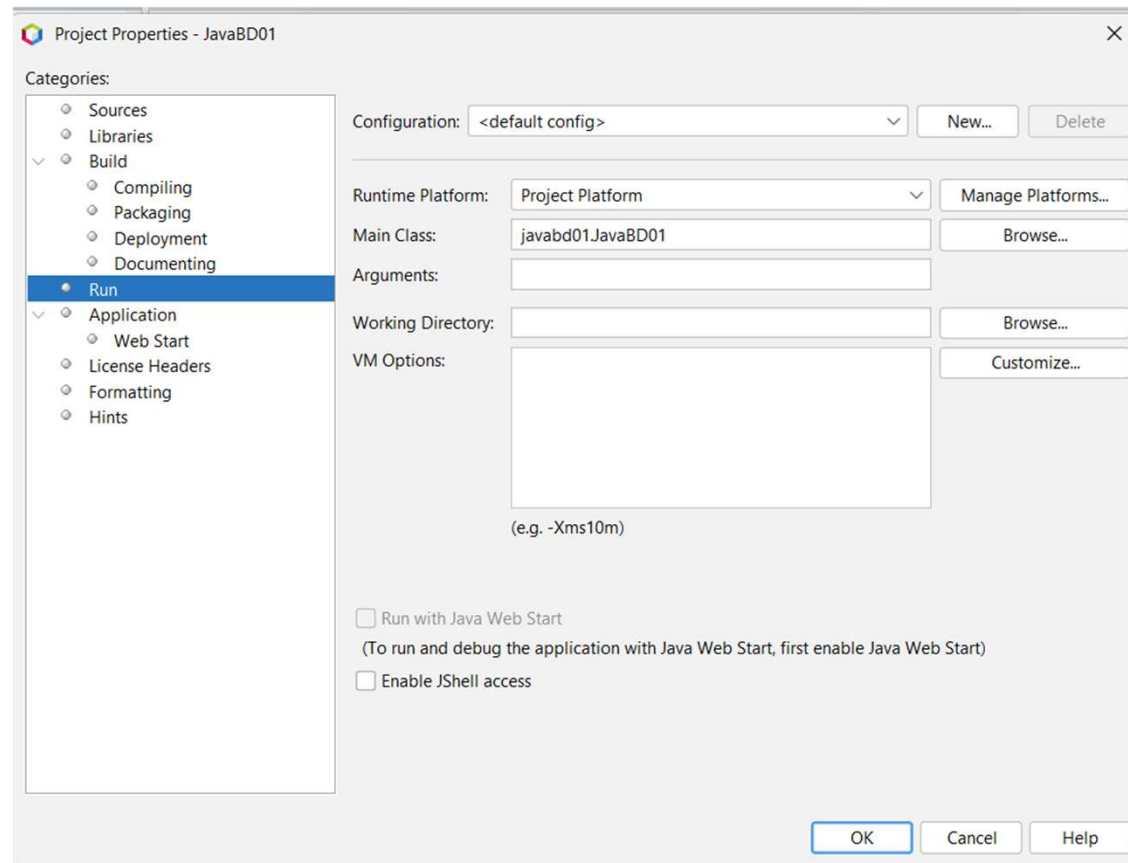
```
private void jMenuItem1ActionPerformed(java.awt.event.ActionEvent evt) {  
    //this.dispose(); - fecha a tela  
    System.exit(status: 0);  
}
```

Projeto com banco MySQL

- Outra coisa que podemos melhorar é definir qual é a classe padrão do nosso programa, ou seja, qual será a classe que será a primeira a ser executada
- Para isso, clique com o botão direito no projeto
- Vá em propriedades

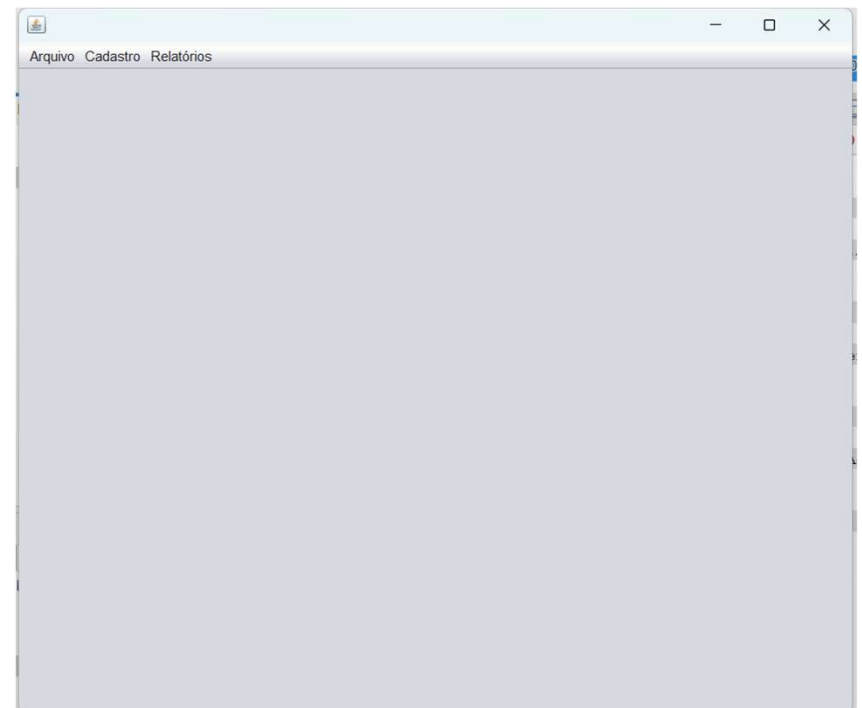
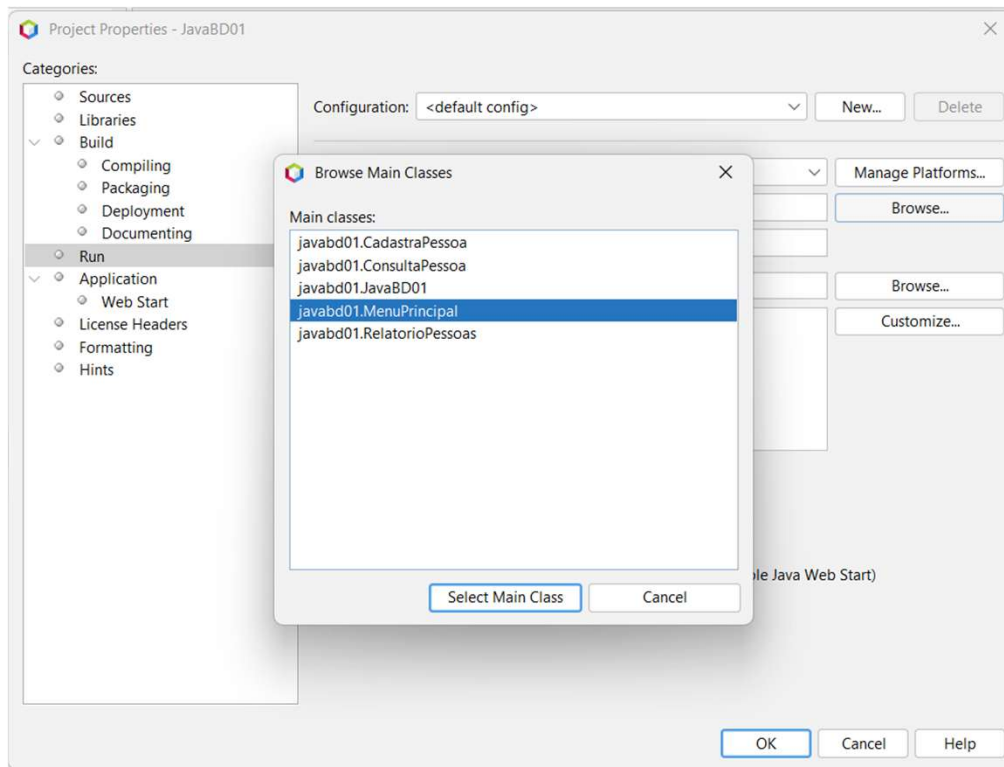


Projeto com banco MySQL

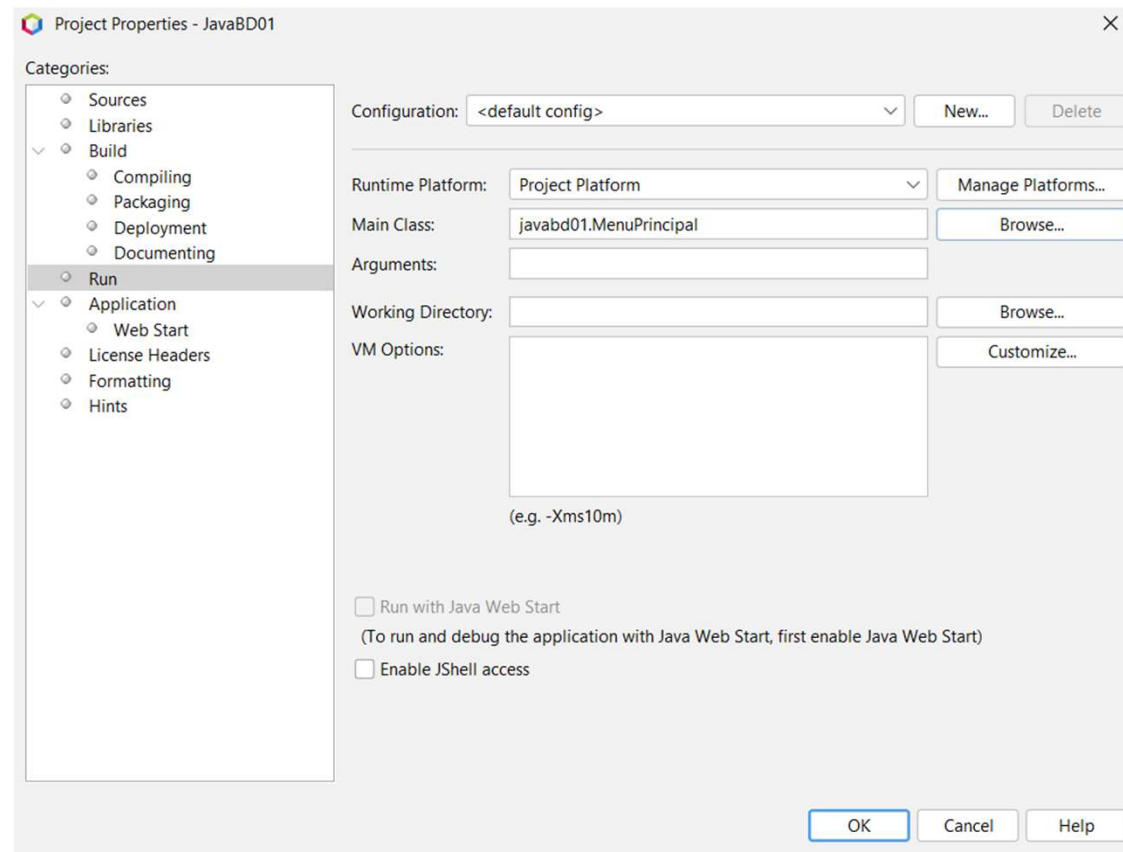


Projeto com banco MySQL

- E compilem:



Projeto com banco MySQL



Agora, vamos fazer uma pesquisa pelo nome:

- Precisamos criar um método que receba um parâmetro para pesquisa e alterar a consulta e adicionar o parâmetro na consulta SQL.

```
public List<Pessoa> getPessoasNome(String nome){
    // "SELECT * FROM pessoa WHERE nome LIKE ?"
    String sql = "SELECT * FROM pessoa WHERE nome LIKE ?";
    try {

        PreparedStatement stmt = conn.prepareStatement(string:sql,i: ResultSet.TYPE_SCROLL_INSENSITIVE,
            i1: ResultSet.CONCUR_UPDATABLE);
        stmt.setString(i: 1, "%" + nome + "%");
        ResultSet rs = stmt.executeQuery(); //obtenho o retorno da consulta e armazeno no ResultSet
        List<Pessoa> listaPessoas = new ArrayList(); //Preparo uma lista de objetos que vou armazenar a consulta
        //Percorre rs e salvar as informações dentro de um objeto Pessoa e depois adiciona na lista
        while (rs.next()){
            Pessoa p = new Pessoa();
            p.setId(id: rs.getInt(string:"id"));
            p.setNome(nome: rs.getString(string:"nome"));
            p.setSexo(sexo: rs.getString(string:"sexo"));
            p.setIdioma(idioma:rs.getString(string:"idioma"));
            listaPessoas.add(e: p);
        }
        return listaPessoas;
    } catch (SQLException ex) {
        System.out.println("Erro ao consultar todas as pessoas: "+ex.getMessage());
        return null;
    }
}
```

Agora, vamos fazer uma pesquisa pelo nome:

```
public List<Pessoa> getPessoasNome(String nome) {  
    // "SELECT * FROM pessoa WHERE nome LIKE ?"  
    String sql = "SELECT * FROM pessoa WHERE nome LIKE ?";  
    try {  
  
        PreparedStatement stmt = conn.prepareStatement(string:sql,i: ResultSet.TYPE_SCROLL_INSENSITIVE,  
            i1: ResultSet.CONCUR_UPDATABLE);  
        stmt.setString(i: 1 ,"%"+ nome +"%");  
        ResultSet rs = stmt.executeQuery(); //obtenho o retorno da consulta e armazeno no ResultSet  
        List<Pessoa> listaPessoas = new ArrayList(); //Preparo uma lista de objetos que vou armazenar a consulta  
        //Percorre rs e salvar as informações dentro de um objeto Pessoa e depois adiciona na lista  
        while (rs.next()){  
            Pessoa p = new Pessoa();  
            p.setId(id: rs.getInt(string:"id"));  
            p.setNome(nome: rs.getString(string:"nome"));  
            p.setSexo(sexo: rs.getString(string:"sexo"));  
            p.setIdioma(idioma:rs.getString(string:"idioma"));  
            listaPessoas.add(e: p);  
        }  
        return listaPessoas;  
    } catch (SQLException ex) {  
        System.out.println("Erro ao consultar todas as pessoas: "+ex.getMessage());  
        return null;  
    }  
}
```

E precisaremos colocar uma entrada de dados na tela

Relatório das Pessoas

Nome:

ID	Nome	Sexo	Idioma
----	------	------	--------

E para fazer a consulta?

- O que precisaremos?
- Como vamos fazer a chamada do método para pesquisar após digitar?

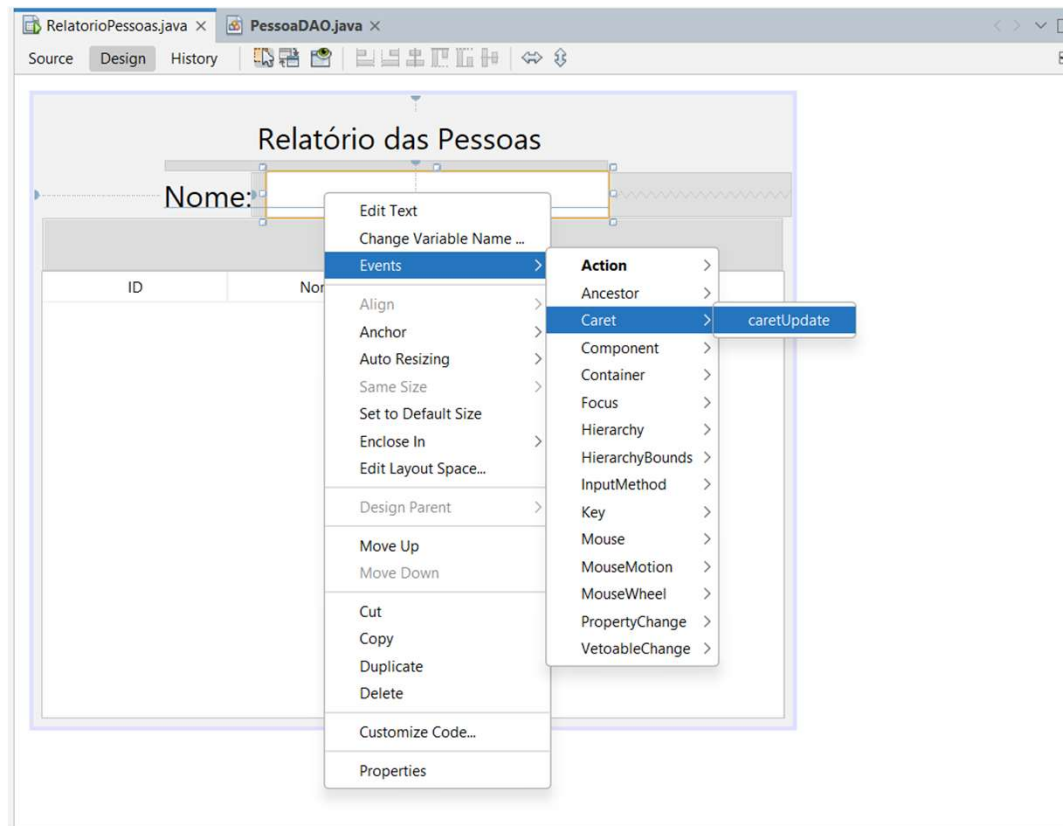
Precisamos alterar o método preencheTabela

```
public void preencheTabela() {
    PessoaDAO pDAO = new PessoaDAO();
    //List<Pessoa> listaPessoas = pDAO.getPessoas();

    List<Pessoa> listaPessoas = pDAO.getPessoasNome(nome: txtNome.getText());
    DefaultTableModel tabelaPessoas = (DefaultTableModel) tbl_Pessoas.getModel();

    //Agora, precisamos percorrer a lista de pessoas que foi consultada e adicionar na tabela:
    for(Pessoa p: listaPessoas){
        Object[] obj = new Object[]{
            p.getId(),
            p.getNome(),
            p.getSexo(),
            p.getIdioma() };
        tabelaPessoas.addRow(rowData: obj);
    }
}
```

Agora, vamos fazer uma pesquisa pelo nome:



Consulta com filtro

- E neste evento, chamaremos o método:

```
private void txtNomeCaretUpdate(javax.swing.event.CaretEvent evt) {  
    preencheTabela();  
}
```

Consulta com filtro

- Funcionou?

Consulta com filtro

- Funcionou?
 - Não né, precisamos limpar a tabela..
 - E como fizemos isso?

Consulta com filtro

```
public void preencheTabela() {
    PessoaDAO pDAO = new PessoaDAO();
    //List<Pessoa> listaPessoas = pDAO.getPessoas();
    List<Pessoa> listaPessoas = pDAO.getPessoasNome(nome: txtNome.getText());
    DefaultTableModel tabelaPessoas = (DefaultTableModel) tbl_Pessoas.getModel();
    //Agora, com a consulta com filtro de nome, para não irmos só adicionando, precisamos limpar a tabela
    // para isso, vamos limpar excluindo todas as alinhas dela
    tabelaPessoas.setNumRows(rowCount: 0);
    //Agora, precisamos percorrer a lista de pessoas que foi consultada e adicionar na tabela:
    for(Pessoa p: listaPessoas){
        Object[] obj = new Object[]{
            p.getId(),
            p.getNome(),
            p.getSexo(),
            p.getIdioma()};
        tabelaPessoas.addRow(rowData: obj);
    }
}
```

Consulta com filtro

- Agora, vamos filtrar pelo sexo?
- É com vocês!

Relatório das Pessoas

Nome:

Sexo: ☐ Masculino ☐ Feminino

ID	Nome	Sexo	Idioma
----	------	------	--------

Consulta com filtro

```
public List<Pessoa> getPessoasNome(String nome, String sexo){
    // "SELECT * FROM pessoa WHERE nome LIKE ?"
    String sql = "SELECT * FROM pessoa WHERE nome LIKE ? and sexo LIKE ?";
    try {
        PreparedStatement stmt = conn.prepareStatement(string:sql,i: ResultSet.TYPE_SCROLL_INSENSITIVE,
            il: ResultSet.CONCUR_UPDATABLE);
        stmt.setString(i: 1 ,"%"+ nome +"%");
        stmt.setString(i: 2 ,"%"+ sexo +"%");
        ResultSet rs = stmt.executeQuery(); //obtenho o retorno da consulta e armazeno no ResultSet
        List<Pessoa> listaPessoas = new ArrayList(); //Preparo uma lista de objetos que vou armazenar a consulta
        //Percorre rs e salvar as informações dentro de um objeto Pessoa e depois adiciona na lista
        while (rs.next()){
            Pessoa p = new Pessoa();
            p.setId(id: rs.getInt(string:"id"));
            p.setNome(nome: rs.getString(string:"nome"));
            p.setSexo(sexo: rs.getString(string:"sexo"));
            p.setIdioma(idioma:rs.getString(string:"idioma"));
            listaPessoas.add(e: p);
        }
        return listaPessoas;
    } catch (SQLException ex) {
        System.out.println("Erro ao consultar todas as pessoas: "+ex.getMessage());
        return null;
    }
}
```

Consulta com filtro

```
public void preencheTabela() {
    String sexo = "";
    PessoaDAO pDAO = new PessoaDAO();
    //List<Pessoa> listaPessoas = pDAO.getPessoas();
    if (rdo_Masculino.isSelected())
        sexo = "M";
    else if (rdo_Feminino.isSelected())
        sexo = "F";
    else
        sexo = "";
    List<Pessoa> listaPessoas = pDAO.getPessoasNome(nome: txtNome.getText(), sexo);
    DefaultTableModel tabelaPessoas = (DefaultTableModel) tbl_Pessoas.getModel();
    //Agora, com a consulta com filtro de nome, para não irmos só adicionando, precisamos limpar a tabela
    // para isso, vamos limpar excluindo todas as alinhas dela
    tabelaPessoas.setNumRows(rowCount: 0);
    //Agora, precisamos percorrer a lista de pessoas que foi consultada e adicionar na tabela:
    for (Pessoa p: listaPessoas) {
        Object[] obj = new Object[] {
            p.getId(),
            p.getNome(),
            p.getSexo(),
            p.getIdioma() };
        tabelaPessoas.addRow(rowData: obj);
    }
}
```

Consulta com filtro

```
private void rdo_MasculinoStateChanged(javax.swing.event.ChangeEvent evt) {  
    preencheTabela();  
}
```

```
private void rdo_FemininoStateChanged(javax.swing.event.ChangeEvent evt) {  
    preencheTabela();  
}
```


Consulta com filtro

Relatório das Pessoas

Nome:

Sexo: ☐ Masculino ☐ Feminino

ID	Nome	Sexo	Idioma
----	------	------	--------

```
private void btn_LimparActionPerformed(java.awt.event.ActionEvent evt) {  
    txtNome.setText("");  
    btnGrp_Sexo.clearSelection();  
}
```